Low-Memory SQIsign through Memory Reuse and Recomputation

Hiro Nakanishi
Independent Researcher
quantumsity@protonmail.com

Preprint: September 5, 2026

Abstract

SQIsign has small public keys and signatures, but its key generation and signing workspaces are not small. In the SQIsign v2.0.1 Level-I Compact implementation, five simultaneously retained regions in `find_uv` alone require 6,339,868 bytes, exceeding RP2350 on-chip SRAM. We combine lifetime-based memory allocation, which shares regions not used simultaneously through typed `union`s, with a representation retaining only the bit length and leading 64 bits of each candidate norm. When both retained fields coincide, the full-precision norms are regenerated from the candidates. The original comparison order is therefore preserved without assuming a collision probability.

The Level-I/RADIX32 operation arena decreases from 353,008 to 172,080 bytes, a reduction of 51.2532%. The v2 firmware uses neither the GNU Multiple Precision Arithmetic Library (GMP), dynamic memory allocation, nor external PSRAM. It reserves a total of 321,408 SRAM bytes and leaves 211,072 bytes unreserved. A differential-conformance test using 100 official input vectors finds matching outputs from the ordinary and low-memory APIs built from the same commit. This is not a known-answer test against historical official responses. Analysis of the linked, fixed firmware image gives Process Stack Pointer (PSP) bounds of 92,192, 120,664, and 20,980 bytes for key generation, signing, and verification, respectively. These values include a maximum 212-byte exception-entry cost and fit within the 131,072-byte PSP reservation. They exclude interrupt handlers, interrupt nesting, and Main Stack Pointer (MSP) use at exception entry.

For v3.0, we apply the same placement principle to three pairs of regions in two functions. The official and adapted implementations each process 300 distinct inputs across three parameter sets once. The Arm stack frames of the two functions decrease in all six comparisons. Processing vector 0 of `p324_3` on RP2350 reduces the Sign PSP watermark from 101,060 to 96,396 bytes, a reduction of 4,664 bytes. This target result covers one parameter set and one deterministic execution path. In both v2 and v3, we detect input-dependent changes in execution time, control flow, and memory addresses. Our conclusions therefore exclude constant-time behavior, power/EM resistance, and production readiness.

Keywords: post-quantum cryptography, SQIsign, microcontroller, low-memory implementation, lifetime analysis, side channels

1 Introduction

NIST selected SQIsign as a Round-3 candidate in its additional post-quantum digital-signature standardization process [1, 2]. SQIsign uses the correspondence between isogenies of supersingular elliptic curves and quaternion ideals to achieve very small public keys and signatures [3, 4, 5]. Small transmitted data are valuable for devices constrained by bandwidth, nonvolatile memory, and certificate size.

Small communication formats and small key-generation and signing workspaces are distinct properties. SQIsign signing combines quaternion arithmetic, four-dimensional lattice operations, norm equations, candidate searches, and multiprecision integer arithmetic. The official v2 implementation assumes GMP and dynamic allocation. Fixed-precision SQIsign [6] and Compact Quaternion Algorithms [7] provide bounds on integer values, but do not address the placement of arrays used simultaneously throughout the protocol. In the Compact implementation used as our starting point, five regions retained by one invocation of `find_uv` alone require 6,339,868 bytes. This is approximately 11.91 times the RP2350's total 532,480 bytes of on-chip SRAM. Even if fixed-precision arithmetic removes the heap, simply moving the same regions into large stack or static arrays does not make the implementation fit on Cortex-M.

For embedded SQIsign, the pqm4 microcontroller benchmarking framework for post-quantum cryptography excluded integration of the 2023 Round-1 submission because of GMP and dynamic allocation [8]. Subsequent research has also primarily targeted verification (Verify) [9, 10]. Fast implementations covering complete v2 key generation (KeyGen), signing (Sign), and verification target host computers or Armv8-A

platforms [11]. In contrast, v3.0, released on September 1, 2026, removes dependencies on external GMP and floating-point libraries and provides complete KeyGen, Sign, and Verify implementations for Cortex-M4 [5, 12]. We therefore evaluate separately whether v2 can execute within on-chip SRAM and whether the same placement principle transfers to the redesigned v3.

We do not change the cryptographic scheme, but redesign the representation, retention, regeneration, and ownership of values. Specifically, we address four research questions.

Q1. Can one deterministic SQIsign v2.0.1 KeyGen, Sign, and Verify path fit within Cortex-M-class on-chip SRAM without GMP, a heap, or external PSRAM?

Q2. Can the original comparison order and cryptographic outputs be preserved without retaining the full-precision candidate-norm table?

Q3. What trade-offs arise among RAM, stack, flash, and execution time? Can input dependencies before and after memory reduction be distinguished?

Q4. Can lifetime-based storage sharing transfer to other large stack frames in SQIsign v3, where the v2-specific candidate search is absent?

We reuse the same physical memory when object lifetimes do not overlap. We also retain each candidate norm's bit length and leading 64 bits, regenerating and comparing full-precision values when both fields agree between the two candidates. The static-arena configuration retaining full-precision candidate norms is the *full-norm-table implementation*; the configuration retaining only bit lengths and leading bits with on-demand recomputation is the *proposed v2 implementation*. For v3, we distinguish the implementation sharing two pairs of regions in one function from the implementation sharing three pairs in two functions. Unless otherwise qualified, the *adapted v3 implementation* refers to the implementation modifying two functions.

Our contributions are as follows.

1. **Protocol-wide lifetime-based memory allocation.** We represent workspaces as objects with types, sizes, alignments, lifetimes, and secrecy attributes. We formulate the problem of sharing regions that are not used simultaneously through a typed arena, including shared storage used sequentially by KeyGen, Sign, and Verify.

2. **Comparing bit lengths and leading bits with full-precision recomputation.** We retain each candidate norm's bit length and leading 64 bits and regenerate its full precision from the candidate only when both fields coincide between the two candidates. We show that the comparator induces the same total order as the original full-precision comparison, and validate it through differential tests including deliberately colliding retained pairs.

3. **A static Cortex-M implementation and evaluation of RAM, stack, flash, and execution-time trade-offs.** For SQIsign v2.0.1 Level I/RADIX32, we remove GMP, dynamic allocation functions, and variable-length arrays (VLAs) from the linked code reachable from KeyGen, Sign, and Verify. The proposed v2 implementation reduces the operation arena from 353,008 to 172,080 bytes (51.2532%), with 321,408 bytes of total SRAM reservation and 211,072 bytes of unreserved SRAM. Flash usage increases by 880 bytes. We validate this configuration using 12 fixed inputs, ordinary/low-memory API differential-conformance tests on 100 official input vectors, PSP analysis of the fixed firmware image, and target measurements.

4. **Evaluation of transfer to SQIsign v3.** We build official v3.0 and the implementation modifying two functions separately and compare them under the same baseline and compilation conditions. The implementation modifying two functions shares three pairs of regions in `quat_lll_dual_reduce_ideal` and `protocols_sign`. We perform host functional tests and Arm stack-frame audits for all three parameter sets. The `p324_3` target test reduces the Sign PSP watermark by 4,664 bytes. We do not transfer the v2-specific bit-length and leading-bit pair, and distinguish the placement principle transferred across versions from v2-specific changes.

5. **Implementation-security assessment.** We evaluate fixed-versus-random timing tests, full-signing control-flow and memory-address differences, and execution-path correspondence with the published simple power analysis (SPA) [13]. We report detected residual input dependencies, without claiming that memory reduction provides constant-time behavior or power/EM resistance.

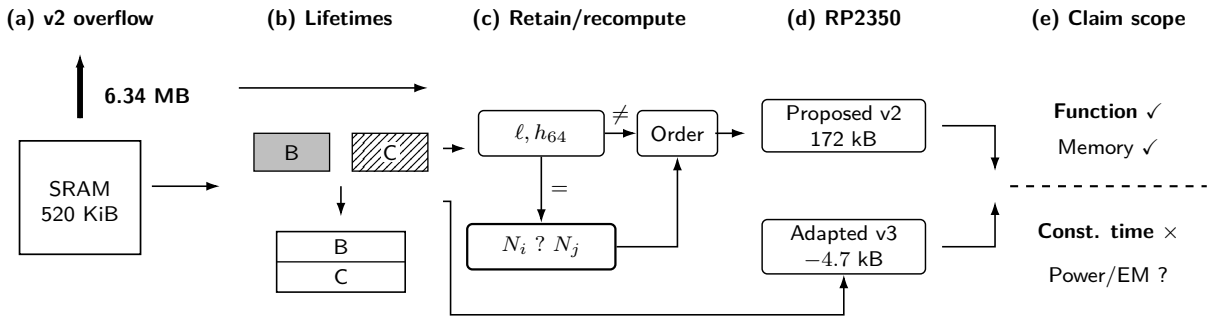

Figure 1: Overview of the study. The proposed v2 implementation combines lifetime-based memory allocation with partial norm storage and recomputation. The adapted v3 implementation transfers lifetime-based memory allocation to two functions for lattice reduction and signing. Both implementations are evaluated on RP2350, with functional and memory evaluation separated from side-channel assessment.

2 Related Work

Sharing storage whose lifetimes do not overlap is not itself new. Modeling simultaneously live values by interference edges is established in graph-coloring register allocation [14]. Compile-time memory allocation for arrays and structures of different sizes has also been studied [15]. We therefore do not claim to invent storage sharing. Our subject is establishing lifetime conditions for complete SQIsign KeyGen, Sign, and Verify, including branches, retries, failure paths, types, alignment, and secret erasure. We combine these conditions with reduced resident data through on-demand full-precision regeneration, and evaluate total SRAM reservation using the linked executable (ELF) and target measurements.

Fixed-precision SQIsign gives uniform bounds of 7026/10713/14150 bits for quaternion intermediates at Levels I/III/V and demonstrates the feasibility of a GMP-free implementation [6]. The current revision of Compact Quaternion Algorithms reduces these bounds to 1665/2521/3319 bits and substantially accelerates host key generation and signing [7]. Our v2 implementation also uses the 1665-bit absolute-value bound at Level I. We take this width as our starting point, but focus on simultaneously used candidate tables and protocol-wide storage placement.

Qlapoti improves IdealToIsogeny, the conversion of quaternion ideals into isogenies, substantially reducing SQIsign execution time and heap usage [16, 17]. The subsequent Qlapoty revisits discrepancies between the published Qlapoti description and implementation and its failure-probability analysis, reporting a new norm-equation algorithm and further SQIsign signing speedups [18]. These advances are incorporated in our starting implementation. However, reported heap usage and host execution time are not Cortex-M total SRAM reservations that include stacks, interrupt reservations, and writable global regions. They cannot therefore serve as like-for-like comparisons with the proposed v2 implementation.

The pqm4 survey of additional signature candidates excluded SQIsign from embedded integration because of GMP and dynamic allocation [8]. Subsequent generation of fast finite-field arithmetic for Cortex-M4 accelerates v2 Level-I Verify, but does not cover KeyGen or Sign [9]. The principal target operation of the one-dimensional SQIsign Cortex-M4 implementation is also Verify [10]. Meanwhile, 32-bit Cortex-A/Linux, Armv8-A NEON, and AVX-512 implementations cover complete KeyGen, Sign, and Verify, but do not target Cortex-M-class on-chip SRAM alone [19, 11, 20]. The pure-Rust `sqisign-rs` is a nearby example of a complete GMP-free implementation, but signing uses `alloc` and `num-bigint`, and it does not report memory usage for all operations on Cortex-M [21]. We thus distinguish three nearest-prior-work groups: complete operations on hosts, GMP-free implementations that use a heap, and Cortex-M implementations limited to Verify.

The official v3 implementation changes this situation. It adopts its own fixed-precision integer layer and provides complete KeyGen, Sign, and Verify on Cortex-M4. Reported Level-I stack usages for these operations are 56, 99, and 37 kB, respectively [5, 12]. We therefore do not claim the first execution of all operations on Cortex-M. We use unmodified official v3 as the control and investigate whether lifetime-based memory allocation reduces signing stack usage under the same source commit and build conditions.

For security, constant-time techniques are advancing separately for integer arithmetic [22], four-dimensional lattice reduction [23], and quaternion operations [24]. None makes the entire current low-memory signer constant-time. The published SPA [13] shows that this gap connects to an actual attack path.

Table 1: Positioning of implementation research directly related to this study

Work	KeyGen/Sign/Verify	Target	Difference from this study
Official v2 [4]	Yes, on hosts	x86/GMP	Assumes heap and GMP
Official v3 [5]	Yes	Includes Cortex-M4	Already fixed precision; control for our v3 evaluation
Fixed precision [6]	Host	x86	Proves value widths; no total simultaneously live SRAM
Compact quaternion [7]	Host	x86	Our starting point; public candidate table exceeds MCU SRAM
Qlapoti/Qlapoty [16, 18]	Host Sign path	x86/GMP	Improves algorithms, heap, and speed; no MCU measurements including stack
pqm4 survey [8]	Excluded from embedded integration	Cortex-M4	GMP and dynamic allocation are barriers
m4-modarith [9]	Verify only	Cortex-M4	No embedded KeyGen/Sign for current v2
`sqisign-rs` [21]	Yes	Host/`no_std`	No GMP, but signing uses heap
32-bit/AVX-512 [19, 20]	Yes	Cortex-A/x86	Host-class memory environment
SQIsign on ARM [11]	Yes	Armv8-A/NEON	Not Cortex-M-class; retains GMP
This study	Yes	RP2350/M33	v2 accounts for all operations' static storage; v3 shares local stack regions

We conducted the literature search on September 4, 2026, including materials published after the v3 release. We found no public example simultaneously demonstrating real v2.0.1 Level-I KeyGen, Sign, and Verify on Cortex-M-class hardware, using on-chip SRAM alone, no GMP, no dynamic allocation, and a whole-firmware total SRAM reservation below 520 KiB. This negative search result is restricted to v2 and is not proof of a world first. Because official v3 already provides complete KeyGen, Sign, and Verify on Cortex-M4, we make no corresponding absence claim for v3. Search targets, queries, exclusion criteria, and the counterexample groups above are fixed in the dated search log at `https://github.com/quantumshiro/tinysqisign/blob/main/results/literature/related-work-search-2026-09-04.md`.

3 Background and Problem Formulation

3.1 Target schemes and execution environment

SQIsign is a signature scheme whose computations use the correspondence between endomorphism rings of supersingular elliptic curves and maximal orders in quaternion algebras [3]. Our primary target, v2.0.1, belongs to the Round-2 specification family and uses chains of $(2, 2)$-isogenies on products of elliptic curves, namely two-dimensional abelian varieties, to compute signature responses [4]. Our v2 experiments therefore do not target a one-dimensional SQIsign implementation. Memory and performance figures for historical Round-1 or one-dimensional verification implementations are not direct comparators. The proposed v2 implementation covers complete Level-I KeyGen, Sign, and Verify paths.

The main computations comprise finite-field and elliptic-curve arithmetic, quaternion-ideal arithmetic, four-dimensional lattice operations, and fixed-precision multiprecision integer arithmetic. Finite-field widths are determined at compile time. On the quaternion side, large intermediate integers, candidate sequences, sorting information, and lattice states may be used simultaneously. Fixed precision eliminates dynamic integer lengths, but applying a uniform maximum width to every routine merely moves large arrays onto the stack or into static storage. Fixed precision alone therefore need not reduce total SRAM.

SQIsign v3.0 is the Round-3 specification released on September 1, 2026 [5]. For `p324_3`, corresponding to Level I, public keys, secret keys, and signatures occupy 83, 270, and 200 bytes, respectively. Version 3 enlarges parameters in response to new attack estimates. It also introduces an inert representation of quaternion ideals, smaller bounds on integer growth, new lattice reduction, ideal-to-isogeny conversion, and a new response distribution. External GMP and the DPE floating-point library are replaced by an in-house fixed-precision integer implementation and computations without floating point. The official source targets x86_64, Arm64, and Cortex-M4 and reports performance and stack usage for complete KeyGen, Sign, and Verify on Cortex-M4 [5, 12].

Version 3 is not a minor modification of v2. The `find_uv` routine and full-precision candidate-norm table responsible for the largest reduction in the proposed v2 implementation do not occur in computations reachable from official v3 source commit `6d017708db403bf83977fa70770fc4f7f9e9ff21` [12]. The v2 bit-length and leading-bit pair therefore has no target in v3. However, v3 still contains large local states with disjoint lifetimes to which lifetime-based memory allocation applies. We evaluate v2-specific data reduction separately from the placement principle usable across versions. The v3 specification explains that the changed response distribution makes the v2 and v3 security assumptions technically incomparable [5]. We likewise do not rank their cryptographic security.

The target is the RP2350 on Raspberry Pi Pico 2. We use one Arm Cortex-M33 core with single-precision floating-point hardware and security extensions [25]. The system clock is 150 MHz, and only the 532,480 bytes

of on-chip SRAM are used. No external PSRAM is used. Code and read-only data are executed and accessed directly from the board's external flash, a mode called execute in place (XIP). The measured firmware is built with Pico SDK 2.3.0 and Arm GNU Toolchain 15.2.1 in Release configuration.

Although RP2350 has more SRAM than typical Cortex-M4 evaluation boards, the original candidate-search workspace is still larger by an order of magnitude. The interrupt MSP, application PSP, shared operation workspace, and runtime data must also occupy the same physical SRAM. We therefore do not evaluate heap usage alone. Our metric is total SRAM reservation, comprising link-time static storage, stack reservations, and the operation arena.

In functional and memory evaluation, failures comprise out-of-bounds accesses, corruption of guards before and after the workspace, unerased secrets, unintended dependencies on dynamic allocation functions or GMP, and output mismatches with the reference implementation. For physical attacks, we assume a strong implementation observer able to observe execution time, control flow, memory addresses, power consumption, and electromagnetic emissions during signing. Our evaluation covers host timing and structural traces, local-operation timing on RP2350, and full-v3-signing execution time. It does not include full-signing power or EM waveforms.

Our functional and memory evaluation concerns output agreement on the targeted paths and conformance to the defined memory boundaries. Secret-independent execution, masking, fault resistance, and production-quality randomness are separate and treated as implementation-security limitations.

3.2 Minimizing simultaneously live memory and conditions for storage sharing

Let $O = \{o_1, \ldots, o_n\}$ be the set of objects used during an operation, and $\mathcal{T}$ the set of admissible execution traces, including inputs, retries, and failure exits. Each o_i has size $s_i \in \mathbb{N}_{>0}$, alignment $a_i \in \mathbb{N}_{>0}$, type τ_i, and secrecy attribute σ_i. Let $L_i(t)$ be the set of instants at which o_i is live in trace $t \in \mathcal{T}$, with $L_i(t) = \varnothing$ when the object does not occur in that trace. Define interference edges by

$$(o_i, o_j) \in E \quad \Longleftrightarrow \quad \exists t \in \mathcal{T} : L_i(t) \cap L_j(t) \neq \varnothing \tag{1}$$

and write this graph as $\mathcal{G} = (O, E)$. Thus, two regions interfere if they can be live simultaneously on any admissible path. Nonoverlap on one successful trace alone does not permit overlay.

For arena offsets $x_i \in \mathbb{Z}_{\geq 0}$, a placement must satisfy

$$x_i \equiv 0 \pmod{a_i}, \tag{2}$$

$$(o_i, o_j) \in E \Longrightarrow (x_i + s_i \leq x_j) \ \vee \ (x_j + s_j \leq x_i). \tag{3}$$

The arena-body usage, excluding surrounding guards, is

$$M_{\text{arena}} = \max_i (x_i + s_i) \tag{4}$$

The minimization problem is an alignment-constrained storage-allocation problem with objective M_{arena}. The proposed v2 implementation does not, however, seek a generally optimal placement. It constructs a feasible placement from manually specified processing phases using C `structs` and `unions`, checked through `sizeof`, `offsetof`, and `_Static_assert`. The arena base address must also satisfy each member's alignment requirements. The implementation does not cast a byte array to arbitrary types. It delegates typed-`union` alignment to the compiler and checks it with an application binary interface (ABI) probe.

M_{arena} is not the SRAM usage of the entire firmware. Let $g_{\text{pre}}, g_{\text{post}}$ be the guards surrounding the arena, $R_{\text{PSP}}, R_{\text{MSP}}$ the PSP and MSP reservations, and R_{other} writable static storage other than the shared operation region and stacks. The whole-firmware total SRAM reservation, counting each region only once, is then

$$R_{\text{excl}} = (M_{\text{arena}} + g_{\text{pre}} + g_{\text{post}}) + R_{\text{PSP}} + R_{\text{MSP}} + R_{\text{other}}. \tag{5}$$

For the proposed v2 implementation, $M_{\text{arena}} = 172{,}080$, the two guards total 128 bytes, $R_{\text{PSP}} = 131{,}072$, $R_{\text{MSP}} = 8{,}192$, and $R_{\text{other}} = 9{,}936$, giving $R_{\text{excl}} = 321{,}408$ bytes. Read-only regions executed directly from flash and unreserved SRAM are not added to this expression.

We do not permit overlay merely on the heuristic expectation that two regions will not be used simultaneously. We impose the following conditions for placing two typed regions at the same offset.

1. Lifetimes do not overlap in the call graph and processing-phase state transitions.
2. Callees do not retain pointers beyond the current phase, and returned values do not reference the region.
3. Secrets are erased before the next phase, with whole-region zero checks where required.

4. C aliasing rules and alignment are ensured through typed `union` members or checked accessors. Byte arrays are not unrestrictedly cast to other types.

5. All exits, including failure paths, check the surrounding guards, secret erasure, and absence of uncommitted values in external outputs.

We check these conditions using ABI size probes, `-Wvla -Walloca -Werror`, AddressSanitizer (ASan), UndefinedBehaviorSanitizer (UBSan), canaries, whole-region erasure tests, and ELF symbol audits. These are not complete static proofs, but detect errors through methods distinct from manually written lifetime annotations.

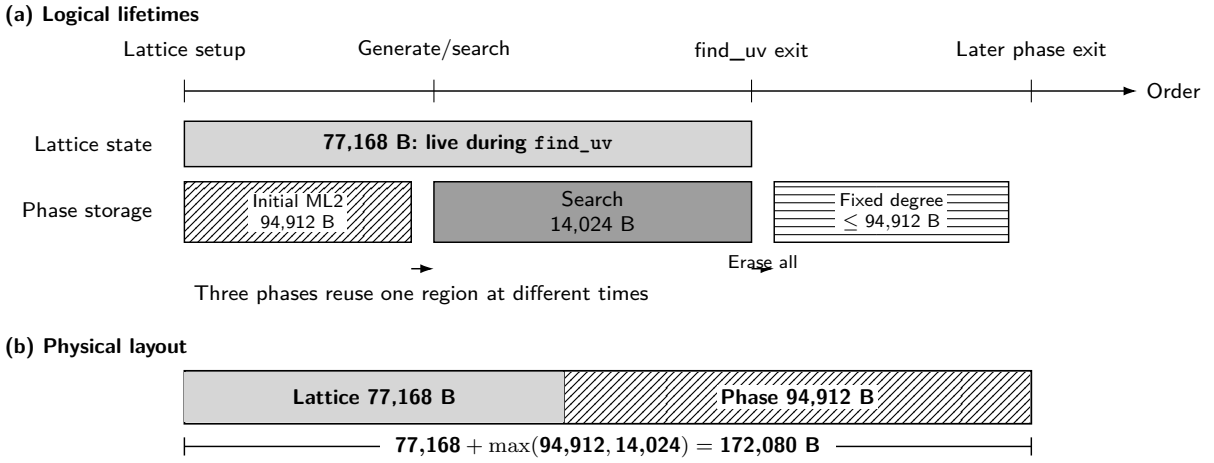

Figure 2: Logical lifetimes and physical placement of the proposed v2 workspace shared along the IdealToIsogeny path. Lattice state remains live throughout `find_uv` and cannot share storage with phase-specific regions such as candidate search. After whole-region erasure at the common `find_uv` exit, fixed-degree processing reuses the same region. Horizontal positions indicate processing order, not relative execution time.

KeyGen, Sign, and Verify do not execute concurrently in this implementation. Their workspaces share one typed `union`, and each operation erases its used regions and the shared arena on exit. This requires reserving only the maximum of the three workspace sizes, rather than their sum. The API is consequently not reentrant and does not permit concurrent signing or simultaneous use from multiple cores. Sequential execution of the three operations on one core is an API precondition.

4 Proposed Method

We combine lifetime-based storage sharing with a representation that regenerates candidate values on demand. Version 2 uses both to reduce resident state, whereas v3 applies storage sharing to two functions. Mapping the placement to C types and annotations makes simultaneously existing values traceable in the implementation.

4.1 Reducing resident state

The starting Compact implementation simultaneously retains candidate vectors, full norms, quotients, row counts, and sorting records within one `find_uv` invocation. We apply the following changes in sequence, checking output agreement after each transformation.

1. Compress four-coordinate candidates proven to lie in $[-2, 2]$ from a wide multiprecision representation into `int8_t[4]`.

2. Sort a `uint16_t` index array instead of duplicating candidates in sorting records.

3. Recompute each candidate's quotient deterministically at its use site instead of retaining it.

4. Replace all-row retention by processing two rows at a time and reusing storage for completed rows.

5. Move large variable-length arrays into caller-owned workspace, reducing dynamic stack-frame records to zero in the targeted Arm code.

6. Share temporary storage for ML2, candidate search, theta chains, discrete logarithms, batch inversion, and multiprecision integers across phases.

7. Consolidate KeyGen, Sign, and Verify workspaces into one sequentially used region.

8. Finally, replace full-precision norm rows by bit-length and leading-bit pairs and on-demand full-precision regeneration.

The earlier candidate compression, indexing, and two-row processing reduce resident data themselves. The later arena construction reuses the same physical storage at different times. Because recomputation may increase flash usage and execution time, we evaluate each transformation using four metrics: RAM, stack, flash, and execution time.

Table 2 identifies each transformation and its accounting scope. Values through two-row streaming cover the `find_uv` candidate-search workspace. Subsequent values are the `sizeof` of caller-owned workspace including lattice state, or of the operation arena. None is a whole-firmware peak. Explicit lattice-state ownership increases the tabulated value by moving 77,168 bytes of variable-length arrays into workspace, without changing total simultaneous memory usage. We retain this configuration in the table to avoid counting a change in storage location alone as memory reduction.

Table 2: Explicitly reserved region sizes after the main transformations. Rows with different accounting scopes are not compared as whole-firmware peak usage.

Configuration	Explicit region [B]	Transformation introduced
Compact starting point	6,339,868	Five simultaneously allocated heap regions
Candidate-coordinate compression	2,044,252	4-byte candidate coordinates and compressed sorting records
Index sorting	1,905,728	Remove candidate copies and reuse a `uint16_t` index array
Deferred quotient recomputation	962,240	Recompute per-candidate quotients at use instead of retaining them
Two-row streaming	275,840	Replace retention of all 7 rows by two rows
Explicit lattice-state ownership	353,008	Move 77,168 bytes of lattice VLAs to typed storage (simultaneous usage unchanged)
Full-norm-table implementation	353,008	Share ML2, theta, discrete-log, and other regions; remove dynamic frames from targeted Arm code
Proposed v2 implementation	172,080	Replace two full-norm rows by retained pairs and regeneration; share ML2 storage

4.2 Comparing bit lengths and leading bits with full-precision recomputation

The norm $N(v)$ of candidate v can be regenerated by evaluating a quadratic form from the retained compressed four-coordinate vector, its row's Gram matrix G_j, and adjusted norm d_j. We use this property to retain only the following pair instead of the full-precision integer $N(v)$ for every candidate. The state G_j, d_j may contain secret-derived information. Regenerability is therefore not evidence of side-channel security.

Definition 1 (Bit length and leading bits using the leading w bits)**.** For a nonnegative integer n, define its bit length $\ell(n)$ as 0 if $n = 0$ and $\lfloor \log_2 n \rfloor + 1$ if $n > 0$. With $w = 64$, define

$$h_w(n) = \begin{cases} n, & \ell(n) \le w, \\ \left\lfloor n/2^{\ell(n)-w} \right\rfloor, & \ell(n) > w \end{cases} \tag{6}$$

and call $S_w(n) = (\ell(n), h_w(n))$ the bit-length and leading-bit pair of n.

The comparator examines (1) bit length, (2) the leading 64 bits, (3) full-precision norms regenerated when retained pairs coincide, and (4) the original enumeration index, in that order.

Proposition 1 (Order preservation)**.** *Suppose $N(v_i)$ can be regenerated at full precision from each candidate v_i. The total candidate order induced by the comparator above agrees with the lexicographic order of $(N(v_i), i)$ obtained by full-precision integer comparison and enumeration-index comparison.*

Proof. If $\ell(N(v_i)) \neq \ell(N(v_j))$, the nonnegative integer with smaller bit length is smaller. If the lengths agree but h_{64} differs, the first differing bit from the most significant end lies within the retained 64 bits, so their order agrees with that of the full integers. If both fields agree, the comparator does not decide from the retained pairs alone, but regenerates and compares both full-precision norms. It compares enumeration indices only if the norms also agree. Thus, all cases agree with the lexicographic order of $(N(v_i), i)$. □

The method does not assume retained pair collisions are sufficiently unlikely. For example, 2^{100} and $2^{100} + 1$ have identical bit lengths and leading 64 bits, but always proceed to full-precision comparison and

return the correct order. Neither cryptographic-hash collision probabilities nor collision frequencies observed on finite input sets are correctness assumptions.

Algorithm 1 gives the procedures corresponding to C functions `finduv_eval_norm`, `finduv_set_norm_sketch`, and `finduv_compare_indices`. FullNorm divides the quadratic-form value by d_j and checks that the remainder is zero and the quotient positive. Rounded approximations are therefore not used to resolve ties.

During sorting, each retained pair tie causes both norms to be regenerated in temporary storage and erased immediately after comparison. During search, the required norms are regenerated at their use sites from candidates addressed by the sorted indices. Initial measurements observed approximately 7,800 sorting comparisons per `find_uv` call. In the memory/functional evaluation corpus, search reevaluations ranged from 2–30 for KeyGen and 2–578 for Sign. A separate fixed-key timing test ran Sign 128 times, including 64 random inputs. In this test, retained pairs coincided in 5,041 of 2,062,361 comparisons, or 0.2444%. Search reevaluations on random inputs ranged from 4–1677. The ranges 2–578 and 4–1677 come from different input corpora. All are performance observations, not worst-case count bounds over all inputs.

Each row is sorted using an in-place heapsort on a `uint16_t` index array of $m \leq 624$ enumeration indices. It uses $O(m \log m)$ comparisons and $O(1)$ auxiliary storage, without recursion or standard-C-library sorting workspace. The C comparison function cannot return an error status directly. If full-precision regeneration fails, it records a `fatal` state and returns enumeration-index order for that comparison only. The status is checked after heapsort and the entire row is rejected, so this provisional order is never exposed as a successful output.

Algorithms 2 to 4 show row generation and sorting, candidate-pair search across two rows, and the schedule preserving the original triangular (j_1, j_2) order while retaining only two rows. Admissible chooses sign representatives, excludes candidates whose coordinates are all divisible by 2 or all divisible by 3, and selects order-4 symmetry representatives for applicable bases, in the original implementation's order. At Level I, $r \leq 7$ and each row has capacity 624. Validating all rows before search prevents an early solution from hiding a malformed later row. SearchPairs searches for solutions for two candidate norms n_1, n_2 satisfying

$$un_1 + vn_2 = T, \qquad u > 0, \quad v > 0 \tag{7}$$

in the same nested-loop order as the original implementation. Specifically, each congruence sequence uses the same scan boundary $0 < v < \lfloor T/n_2 \rfloor$. The first solution here means the first accepted solution within this reference scan domain and order, not a new completeness claim over all positive integer solutions. The additional sum-of-two-squares condition is disabled in calls from the proposed v2 implementation. During search, n_1, n_2 are regenerated from compressed candidates, and only selected candidates are copied to external outputs. This search has a variable iteration count and is not constant-time.

Algorithm 1 Norm comparison with full-precision fallback when required

Input: Compressed candidate v, Gram matrix G, nonzero adjusted norm d
Output: On success, a positive full-precision norm, a 10-byte retained pair, or comparison of $(N(v_i), i)$

function FullNorm(v, G, d)
  $p \leftarrow$ WidenToIbz(v)
  $q \leftarrow$ QuadraticForm(p, G) ▷ Evaluate $q = p^{\mathsf{T}}Gp$ at fixed precision
  $(n, r) \leftarrow$ DivRem(q, d)
  if $r \neq 0$ **or** $n \leq 0$ **then**
    return Fatal
  end if
  return n
end function
function MakeSketch(n)
  $\ell \leftarrow \lfloor \log_2 n \rfloor + 1$
  if $\ell >$ `UINT16_MAX` **then**
    return Fatal
  end if
  $h \leftarrow n$ **if** $\ell \leq 64$ **else** $\lfloor n/2^{\ell-64} \rfloor$
  return (ℓ, h) ▷ ℓ: 16 bits; h: 64 bits
end function
function Compare(i, j, V, S, G, d)
  if $i = j$ **then**
    return 0
  else if $S[i].\ell \neq S[j].\ell$ **then**
    return Cmp($S[i].\ell, S[j].\ell$)
  else if $S[i].h \neq S[j].h$ **then**
    return Cmp($S[i].h, S[j].h$)
  end if
  $n_i \leftarrow$ FullNorm($V[i], G, d$); $n_j \leftarrow$ FullNorm($V[j], G, d$)
  if $n_i =$ Fatal **or** $n_j =$ Fatal **then**
    return Fatal
  else if $n_i \neq n_j$ **then**
    return Cmp(n_i, n_j)
  else
    return Cmp(i, j) ▷ Total order via original enumeration indices
  end if
end function

Algorithm 2 Generating a candidate row and sorting it in full-precision order

Input: Row j, resident-row slot, caller-owned workspace W
Output: Compressed candidate row sorted by full-precision key $(N(v), i)$ and count m, or Fatal

function PrepareRow(j, slot, W)
  $m \leftarrow 0$
  for $v \in [-2, 2]^4$ in the original implementation's order **do**
    if not Admissible(v, G_j) **then**
      continue
    end if
    $n \leftarrow$ FullNorm(v, G_j, d_j)
    if $n =$ Fatal **then**
      return Fatal
    end if
    if $n \bmod 2 = 1$ **then**
      if $m = 624$ **then**
        return Fatal
      end if
      $W.V[\text{slot}, m] \leftarrow$ Pack4xInt8(v)
      $s \leftarrow$ MakeSketch(n)
      if $s =$ Fatal **then**
        return Fatal
      end if
      $W.S[m] \leftarrow s$; $m \leftarrow m + 1$
    end if
  end for
  $P[k] \leftarrow k$ for $0 \le k < m$
  HeapSort(P, comparator Compare)
  if a comparison error was recorded **then**
    return Fatal
  end if
  if not ApplyPermutationByCycles($W.V[\text{slot}], P$) **then**
    return Fatal
  end if
  return m
end function

Algorithm 3 Candidate-pair search over two sorted rows preserving reference scan order

Input: Target $T > 0$, sorted rows $A[0..m_1 - 1], B[0..m_2 - 1]$, row descriptions $R_k = (G_k, d_k)$, diagonal flag δ
Output: The first $(u, v, i_1, i_2, n_1, n_2)$ satisfying equation (7) within the reference scan domain, NotFound, or Fatal

function SearchPairs($T, A, B, R_1, R_2, \delta$)
  for $i_1 \leftarrow 0$ **to** $m_1 - 1$ **do**
    $n_1 \leftarrow$ FullNorm($A[i_1], R_1.G, R_1.d$)
    if $n_1 =$ Fatal **then**
      return Fatal
    end if
    $a \leftarrow T \bmod n_1$; $s \leftarrow i_1$ **if** δ **else** 0
    for $i_2 \leftarrow s$ **to** $m_2 - 1$ **do**
      if δ **and** $i_1 = i_2$ **then**
        $n_2 \leftarrow n_1$
      else
        $n_2 \leftarrow$ FullNorm($B[i_2], R_2.G, R_2.d$)
        if $n_2 =$ Fatal **then**
          return Fatal
        end if
      end if
      if $n_2^{-1} \bmod n_1$ does not exist **then**
        continue
      end if
      $v \leftarrow a\, n_2^{-1} \bmod n_1$; $q \leftarrow \lfloor T/n_2 \rfloor$
      while $v < q$ **do**
        $(u, \rho) \leftarrow$ DivRem($T - vn_2, n_1$)
        if $\rho \neq 0$ **or** $u \leq 0$ **then**
          return Fatal
        end if
        if $v > 0$ **then**
          return $(u, v, i_1, i_2, n_1, n_2)$
        end if
        $v \leftarrow v + n_1$
      end while
    end for
  end for
  return NotFound
end function

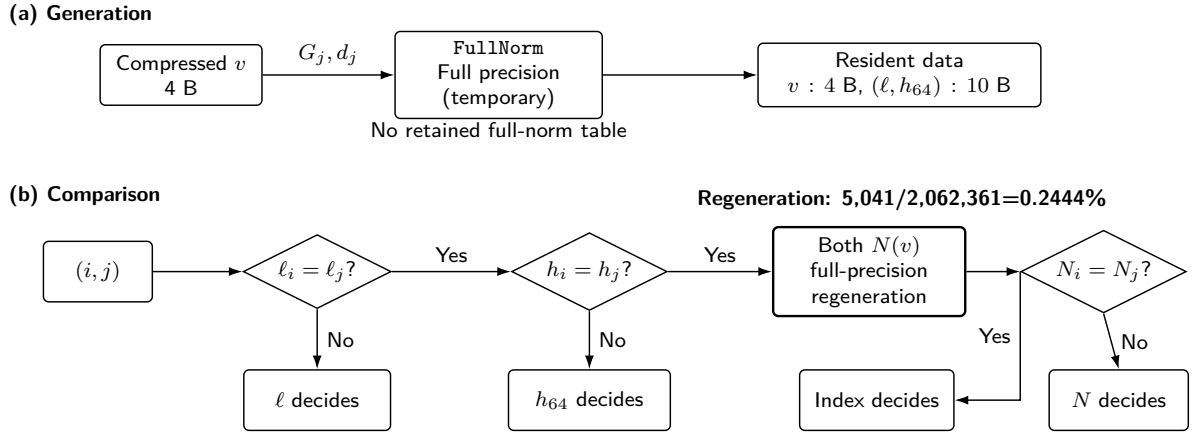

Figure 3: Storage format for bit lengths and leading bits and comparison paths. Different bit lengths or leading 64 bits determine the order from retained pairs alone. When both agree, full-precision norms are regenerated from compressed candidates. The regeneration rate shown is observed in the fixed-key timing test; comparison correctness does not depend on this frequency.

Algorithm 4 Proposed v2 `find_uv`: validate all rows before searching two at a time

Input: Target T, base ideal and quaternion ring $\mathcal{I}$, r quaternion orders, caller-owned workspace W
Output: The same first solution as the original, NotFound, or Fatal; $W = 0$ at every exit

function FindUV-LM($T, \mathcal{I}, W$)
  if arguments or fixed capacities are invalid **then**
    SecureClear(W);
    return Fatal
  end if
  SecureClear(W)
  $L \leftarrow$ PrepareLatticeState($T, \mathcal{I}, W$.phase.lattice_mul)
  if $L =$ Fatal **then**
    return Finish(Fatal, W)
  end if
  $R_j \leftarrow (L.\text{gram}[j], L.\text{adjusted\_norm}[j])$ for $0 \le j < r$
  SecureClear(W.phase) ▷ One-way transition to candidate processing
  for $j \leftarrow 0$ **to** $r - 1$ **do** ▷ Validate all rows, reusing slot 0
    $c[j] \leftarrow$ PrepareRow($j, 0, W$)
    if $c[j] =$ Fatal **then**
      return Finish(Fatal, W)
    end if
  end for
  for $j_1 \leftarrow 0$ **to** $r - 1$ **do**
    $m_1 \leftarrow$ PrepareRow($j_1, 0, W$)
    if $m_1 \neq c[j_1]$ **then**
      return Finish(Fatal, W)
    end if
    for $j_2 \leftarrow j_1$ **to** $r - 1$ **do**
      if $j_2 = j_1$ **then**
        $(V_2, m_2) \leftarrow (W.V[0], m_1)$ ▷ Reuse the same row on the diagonal
      else
        $m_2 \leftarrow$ PrepareRow($j_2, 1, W$); $V_2 \leftarrow W.V[1]$
        if $m_2 \neq c[j_2]$ **then**
          return Finish(Fatal, W)
        end if
      end if
      $V_1 \leftarrow W.V[0][0..m_1 - 1]$
      $z \leftarrow$ SearchPairs($T, V_1, V_2, R_{j_1}, R_{j_2}, j_1 = j_2$)
      if $z =$ Fatal **then**
        return Finish(Fatal, W)
      end if
      if z is a solution **then**
        if not ValidateAndPublish(z) **then**
          return Finish(Fatal, W)
        end if
        return Finish(Success, W)
      end if
    end for
  end for
  return Finish(NotFound, W)
end function

Proposition 2 (Preservation of the first solution)**.** *For a fixed input, assume all full-precision arithmetic succeeds in both the reference and proposed v2 implementations. If both sort each row by $(N(v), i)$ and scan pairs of quaternion orders, within-row indices, and congruence solutions over identical domains and in identical orders, the proposed v2 implementation returns the same first solution as the reference. If no solution is accepted within the reference scan, both return* NotFound*.*

Proof. By Proposition 1, row order obtained from retained pairs and full-precision regeneration agrees with that of the full-norm rows. Initial validation carries no state other than counts into search, and each row is regenerated by the same procedure during search. The diagonal start $i_2 = i_1$, off-diagonal start $i_2 = 0$, triangular (j_1, j_2) order, and congruence sequences in Algorithm 3 all agree with the reference scan. Thus, if a solution is found, both implementations accept the same first combination; otherwise, both return NotFound. □

To connect the premises of Proposition 2 to actual C code, we compare `dim2id2iso.c` in the full-norm-table implementation retaining full-precision norm rows (commit `6b79cfb5...`) and the immediately subsequent proposed v2 implementation (commit `71099e08...`). Table 3 gives the correspondence. The checking script verifies both source digests and checks 22 conditions concerning enumeration, comparison, scanning, and failure handling. This connects the scan domain and order assumed in the proposition to the C implementations in these two commits.

Table 3: Correspondence of `find_uv` scans in the full-norm-table reference and proposed v2 implementation

Check	Full-table reference	Proposed v2	On failure
Row generation	`x/y/z/w`, sign representative, 2/3 exclusion, order-4 representative, odd norm	Same loops and predicates, then `finduv_eval_norm`	Nonintegral quotient, nonpositive value, or excess capacity: Fatal
Sort key	Full-precision $(N(v), i)$	Bit length, leading 64 bits, full-precision regeneration, i	Record regeneration failure; reject row as Fatal
Quaternion-order pairs	Increasing j_1, $j_2 \geq j_1$	Same triangular order; same row on diagonal	Regenerated count mismatch: Fatal
Within-row indices	Increasing i_1; diagonal $i_2 = i_1$, otherwise $i_2 = 0$	Same order in `find_uv_from_compact_lists`	Norm regeneration failure: Fatal
Congruence sequence	$v = (T \bmod n_1)n_2^{-1} \bmod n_1$, $v{+}{=}n_1$	Same representative, increment, $v < \lfloor T/n_2 \rfloor$	Noninvertible: next candidate; corrupted arithmetic: Fatal
Output/exit	Validate ideals and output only after success	Same output order; additional condition disabled on this path	Separate Success/NotFound/Fatal; erase whole workspace

In addition to this audit, we perform differential tests between the full-table reference and proposed v2 implementation on 12 fixed inputs, and between ordinary and low-memory APIs on 100 official input vectors. We separately test deliberate retained pair collisions, NotFound, retries, Fatal, guards, and erasure paths. These are not formal proofs of whole-C-program semantics or all-input behavior. Correspondence tables, source locations, and machine-check results are provided at `https://github.com/quantumshiro/tinysqisign/tree/main/results/revision-2026-09-04`.

Here, FINISH denotes the common exit that finalizes initialized fixed-precision objects and then erases the entire workspace with `sqisign_secure_clear`, regardless of success, failure, or no solution found. In the implementation, long-lived lattice state occupies 77,168 bytes and phase-shared storage 94,912 bytes, including a 14,024-byte candidate-search region. Thus, the `find_uv` type size is

$$77{,}168 + \max(94{,}912, 14{,}024) = 172{,}080 \text{ bytes} \tag{8}$$

in total.

4.3 Local lifetime-based memory allocation in the v3 adaptation

Compiling official v3 `p324_3/m4f` for RP2350 gives a largest reported fixed-size function frame of 34,816 bytes in `quat_lll_dual_reduce_ideal`. This function contains two pairs of regions whose lifetimes do not overlap across sequential computations.

The first overlay places lattice-reduction state `fp_ldl_t` and the 4×4 integer matrix `Aout`, constructed only after reduction, in one typed `union`. Because a pointer to `fp_ldl_t` is passed to the reduction routine, the compiler does not necessarily reuse its stack location automatically. We make the disjoint lifetimes explicit through C type layout so that this reuse is reflected in the linked binary.

The second overlay reuses the now-unneeded inverse-transform matrix `Ainv_total` in `ct_mk_result_t` as basis-output storage `Bout`, after deriving `Aout` from it. The Gram matrix `G` and transform `T` in the same structure remain live until later norm computation and are not shared. A `_Static_assert` checks that `Ainv_total` and `Bout` have equal storage sizes.

The third overlay applies to the signing body `protocols_sign`. The commitment ideal is no longer referenced after construction of `ideal_skchall_aux` and the greatest-common-divisor check. The output product of the challenge isogeny is first written afterward. Placing these two objects in `protocols_sign_phase_t` reduces the function's `p324_3` stack frame from 20,008 to 19,272 bytes. A lifetime audit table recording source locations checks that the commitment ideal's last reference precedes the first write to the output product.

The implementation modifying two functions applies these three changes to two functions. Algorithms, branch conditions, randomness consumption, and public APIs remain unchanged; only local-variable ownership and placement change. This is an application of equations (1) and (3) to two functions, not a data-representation change such as the v2 bit-length and leading-bit pair. We use the implementation modifying one function for repeated target and binary-layout tests, and the implementation modifying two functions to evaluate transfer of lifetime-based memory allocation to two functions.

4.4 Generating placement from lifetime annotations

To generate typed placement from lifetime conditions, we annotate each object with `phases` (processing phases using it), `size`, `alignment`, `secret` (secrecy), `pointer_escape` (pointer retention), and `clear_on` (erasure points). The generator creates interference edges between objects whose phase sets intersect, then assigns offsets using alignment-constrained first fit. It outputs a JSON layout, an interference graph, typed C `union`s and `struct`s, and `_Static_assert`s.

The v2 `find_uv` annotations express interference between the 77,168-byte long-lived lattice state and three phase-specific objects. Sharing the latter three at offset 77,168 generates a total layout of 172,080 bytes. We compile the generated header with the actual Cortex-M33/RADIX32 types and confirm agreement with the manually implemented `find_uv_workspace_t` size and offsets. We additionally enumerate 4 physical objects in the C header and 18 forms of direct access to members or entire phase regions in the source. Unannotated members or direct accesses cause the check to fail.

Automation covers interference-graph generation and placement from supplied annotations, and coverage checks for direct accesses. It does not infer phase semantics, lifetimes, indirect aliases, pointer retention, erasure on all failure paths, function-pointer targets, post-link-time-optimization call graphs, or absence of uncommitted external outputs from C source. Placement validity therefore depends on annotation completeness. Separate checks using lifetime audit tables, dynamic canaries, erasure tests, and ELF audits remain necessary.

5 Implementation and Evaluation Method

Evaluating the proposed method requires fixing integer representation widths and physical memory placement, and matching build conditions between compared implementations. Functional tests examine output agreement; memory checks examine storage sharing, erasure, and conformance to reservations.

5.1 Integer representation and memory placement

We start from the public Compact Quaternion Algorithms implementation and fix SQIsign v2.0.1 Level I with RADIX32 in the finite-field and curve layers. RADIX32 denotes the finite-field radix representation, not the quaternion-integer word width. Quaternion-layer `ibz_t` is `uint64_t[27]`, obtained by adding a sign bit to the 1665-bit absolute-value bound and rounding up to a 64-bit word boundary. This provides 1728 bits of signed storage, requiring 216 bytes per integer. Multiprecision multiplication, division, greatest common divisors, square roots, and modular exponentiation operate on these fixed arrays and dedicated workspace. The finite-field and curve layers also use existing fixed-width implementations.

This choice fixes every integer object's capacity and alignment at compile time, allowing direct inclusion in the placement of equations (1) and (3). Fixed width does not, however, imply a fixed operation count. Scans of currently used words, comparisons, Euclidean algorithms, and exponentiation are variable-time. We do not claim constant-time integer processing. The proposed v2 SQIsign source is pinned to commit `71099e0827d3f0a3b3c705d2eda592c401e0d57d`, and the RP2350 firmware to `dc3289add3213cc7671f9943dfaa3bac770b2709`.

According to the official GMP 6.3.0 manual, `mpz_t` holds an integer size and a pointer to allocated data. Insufficient capacity triggers further allocation, using `malloc`, `realloc`, and `free` by default [26]. Ordinary GMP therefore allocates and reallocates multiprecision limbs on a general heap. This does not meet our requirements of bare-metal code without dynamic allocation functions and compile-time-determined total SRAM reservation. mini-GMP can reduce dependencies, but the bundled version used here also allocates variable-length limb arrays per `mpz_t`. Linking unmodified mini-GMP therefore does not resolve memory ownership.

For functional tests, we change only the integer backend in the Level-I implementation at official source commit `dd133d7aca576c361a270c8e6434832535b42ecc`. System GMP, unmodified mini-GMP, and a static-pool mini-GMP variant with erasure all pass the official 100-vector known-answer test. On the fixed LP64 host input corpus, however, unmodified mini-GMP signing requests at most 67,168 bytes of simultaneously live limbs, but up to 4,830 simultaneously live allocations. With a simple 16-byte-aligned first-fit pool allocator, the smallest pool completing all 100 paths is 317,696 bytes; 317,680 bytes exhausts it. This measurement depends on allocator, ABI, and input corpus. It is neither an inherent mini-GMP lower bound nor a Cortex-M33 upper bound. The pool excludes the official implementation's stack, other writable state, and MSP, and is therefore not directly compared with the proposed v2 whole-firmware value of 321,408 bytes.

For the same official host source, the median-time ratios of unmodified mini-GMP to system GMP are 1.3621 for KeyGen and 1.4497 for Sign. The simple pool allocator with erasure on free takes 4.9441 times the KeyGen time and 4.2316 times the Sign time of allocation-recording mini-GMP. These measurements

include the execution time of our allocator implementation and do not characterize mini-GMP in general. In isolated operations, mini-GMP is faster for greatest common divisors and inverses, whereas the fixed-width implementation is faster for the measured multiplications, divisions, and square roots. Before unused-section removal, instruction regions in Arm GNU 15.2.1 object code compiled with `-Os` occupy 7,962 bytes for fixed width and 18,566 bytes for mini-GMP. Our main reason for choosing fixed width is not speed, but the ability to audit the following properties over all reachable paths.

1. No heap is used on any reachable path, and the operation reservation can be fixed before linking.
2. Fragmentation, metadata, and deallocation order need not be separately analyzed for thousands of variable-length regions.
3. The whole shared region can be erased at operation exit and checked for unerased freed limbs.
4. No additional input-dependent traces of allocation sizes, counts, or relocation addresses are introduced.

Value-dependent control flow and memory addresses remain in the fixed-width implementation. The last item therefore means only that it removes the additional leakage surface associated with dynamic allocation. Capacity-limited mini-GMP with a dedicated allocator is another possible design. We exclude it from our comparison because pool bounds covering every retry path, total SRAM on 32-bit hardware, erasure, cycle counts, and leakage tests have not been evaluated.

The proposed v2 operation arena body occupies 172,080 bytes; including surrounding guards, the shared operation region occupies 172,208 bytes. The main SRAM bank holds this region, a 131,072-byte PSP reservation, and 9,936 other bytes, totaling 313,216 bytes reserved at link time. Another 8,192 bytes in a separate auxiliary bank are reserved for MSP. The total SRAM reservation is therefore

$$313{,}216 + 8{,}192 = 321{,}408 \text{ bytes} \tag{9}$$

leaving 211,072 bytes unreserved.

For v3, official source commit `6d017708db403bf83977fa70770fc4f7f9e9ff21` is the unmodified control. The implementation modifying one function is pinned to `9293313fb58de4c5ce9dd27a5a9fde0058766c79`, and the implementation modifying two functions to `874658c64aa2e20f53b1f4d696144723d558ed5c`. The latter changes only `lll_dim4.c` and `sign.c`. For all three official parameter sets, we bind the generating commit to the directory-tree digest of generated `pqm4` source. The RP2350 implementation modifying two functions is built from firmware commit `5df7acfb2e32019305094546939bdc50bbf71e00`, using Pico SDK 2.3.0, Arm GNU Toolchain 15.2.1, Release configuration, 150 MHz, and one core. Reconstruction patches and digests are included in the public supplementary artifacts.

Official v3 places large local state primarily on the stack. The proposed v2 arena body and v3 PSP watermark are therefore different quantities. The v3 firmware reserves 131,072 bytes for the application PSP, fills it with a known pattern immediately before each operation, and measures the contiguous overwritten depth afterward. The interrupt MSP reservation is 8,192 bytes.

Both official and adapted v3 have zero-byte heap sections in their linked ELFs. However, newlib allocation-related symbols remain through diagnostic `fprintf`/`abort` paths, and GCC stack-usage output contains 19 dynamic-frame records. For the adapted v3 implementation, we therefore claim only that no heap use was observed on successful KeyGen, Sign, and Verify paths and that the targeted functions' local stack frames shrink. Unlike proposed v2, we do not claim removal of every allocation-related symbol and dynamic frame from the linked reachable scope.

5.2 Comparison conditions and repetition design

We divide evaluation into five groups.

1. **Functional equivalence**: byte agreement of public keys, secret keys, signed messages, and post-operation random-generator state with reference paths; acceptance of valid signatures and rejection of modified signatures.
2. **Memory**: type `sizeof`, linker maps, ELF sections, heap policy, PSP/MSP watermarks, and unreserved SRAM.
3. **Portability**: relevant RADIX64/RADIX32 and Level-I/III/V tests, Arm compilation, C aliasing rules, and prohibition of VLAs and `alloca`.
4. **Performance**: host repetitions with reversed execution order and per-operation elapsed time on RP2350; results from these environments are kept separate.

5. **Input dependence**: fixed-versus-random timing tests, reproducibility of control-flow and effective-address sequences, and local modular-exponentiation timing diagnostics.

The proposed v2, official v3, and adapted v3 implementations are independently built from pinned commits, without sharing object files. Comparison manifests bind commits, ELFs, target captures, and measurement definitions using SHA-256.

Official and adapted v3 use the same baseline commit, parameter set, and compiler options. Source differences are restricted to one file for the implementation modifying one function and two files for the implementation modifying two functions. We use these within-version comparisons to estimate local storage-sharing effects. Between v2 and v3, parameters, algorithms, integer representations, and ownership models change simultaneously, so cross-version figures are reported only for context. In particular, we do not subtract the explicit v2 arena and the v3 stack watermark as though they were one RAM metric.

The v2 target test uses a deterministic test random generator based on AES-256 CTR-DRBG, with the same seed, message, and expected digests as the host reference. This source is for differential testing, not production entropy. Manifests record the clock, toolchain, linker layout, binary hash, and capture hash. We boot the same UF2 firmware image twice, execute KeyGen, Sign, and Verify sequentially, and check guards and whole-region erasure after each operation. This repeats one input rather than broadening the input distribution.

The v2 differential-conformance test uses 100 archived NIST-v2 Level-I official input vectors. Their input-file SHA-256 is `81ff60e3...d6ebc7e`, with the full value in the supplementary artifacts. We separately build ordinary and `SQISIGN_LOWMEM_ONLY` APIs from the same commit and supply identical inputs. The low-memory build does not call ordinary entry points; it runs only caller-workspace KeyGen, Sign, Open, and Verify. Each vector checks recovery of a valid signed message, rejection of a one-bit signature modification, guards, and post-operation erasure. The ordinary API runs once, and the low-memory API twice in separately launched processes. The three response sequences agree, but share the same input corpus and ordinary path from the same commit, so they are not independent expected values. Nor is this a known-answer test against historical official responses.

The v3 target test embeds official known-answer vector 0 in firmware and compares public keys, secret keys, signed messages, and recovered messages byte for byte. We compare the implementation modifying one function with official v3 in five pairs with reversed execution order, totaling ten boot-time measurements. The implementation modifying two functions measures KeyGen, Sign, and Verify once on vector 0. Reported measurements are pinned to source and firmware without uncommitted changes. SHA-256 digests of binaries, captures, and patches are recorded in the public supplementary artifacts.

On the host, we build official v3 and the implementation modifying two functions separately. For each of `p324_3`, `p500_27`, and `p664_17`, we run API tests, scheme self-tests, and 100 official responses. There are 3 sets $\times$ 100 vectors, or 300 distinct parameter/input pairs. Each pair runs once on official v3 and once on the implementation modifying two functions, totaling 600 checks. Arm stack-frame audits cover the same six configurations, without shared object files across implementations.

Separately from single-vector repetitions, we build firmware processing official response vectors 0–9 sequentially in one boot. For official v3 and the implementation modifying one function, we use two layouts placing either 0 or 512 Thumb no-operation (NOP) instructions immediately before the cryptographic library. The latter shifts the entry addresses of `crypto_sign_keypair`, `crypto_sign`, and `crypto_sign_open` by 1,024 bytes. In addition to agreement of valid KeyGen, Sign, and Verify outputs, every vector must reject a modified signature with its first bit flipped.

5.3 Functional and memory-safety validation

For the measured firmware, we fix 52 project translation units reachable from Level-I/RADIX32 KeyGen, Sign, and Verify. ELF audits check the absence of dynamic allocation functions, GMP, system random generators, legacy large-stack paths, and unresolved symbols. They also check that placed sections lie within internal SRAM or XIP flash. Heap reservation is zero bytes. Guards surround the shared operation region, and we verify that the entire workspace is zero after each operation.

This method does not rely merely on the absence of visible `malloc` calls in source. The linked ELF is checked for allocation inside libraries, retained legacy paths, implicit VLAs, and unaccounted linker-script regions.

To make sharing safety traceable, we construct a lifetime audit table for 11 major regions. Each row records object name, type, Arm-ABI size/alignment/offset, construction and last-reference sites, mutually exclusive partners, pointer passing and retention by callees, erasure sites on normal/failure/retry paths, and corresponding static/dynamic checks. Table 4 summarizes it; the complete CSV and check results are public supplementary artifacts.

Table 4: Summary of lifetime audit tables for major shared v2 regions

Object (type)	Size [B]	Align. [B]	Mutually exclusive partner	Exit/check
Shared operation region	172,080	8	Sequential KeyGen/Sign/Verify	Whole-region zero scan after each operation; guards
KeyGen shared region	172,080	8	`find_uv` and discrete log	Erase at all exits; callee retry conditions; static assertions
Sign shared region	172,080	8	7 main phases	Erase at all exits; guards; 100-input differential test
`find_uv` lattice state	77,168	8	No sharing: live during candidate search	Struct concatenation; offset assertions; ASan/UBSan
`find_uv` phase region	94,912	8	Lattice multiplication, candidate search, fixed-degree processing	One-way transitions; erase all exits including Fatal/NotFound
Verify shared region	15,428	4	Sequential even-degree isogeny and theta	Erase all wrapper exits; modified-signature tests

To investigate pointer retention beyond intended lifetimes, we mechanically scan explicit stores to global or static variables in the target source. We find no workspace pointer retained outside synchronous calls. This is not an analysis capable of detecting every alias. In particular, outer Sign retries do not erase all 172,080 bytes on every iteration. They rely on finalization/erasure by active callees and initialization/overwrite by the next phase. Whole-operation-region erasure is directly confirmed at the final public workspace-API exit. The CSV documenting this limitation is at `https://github.com/quantumshiro/tinysqisign/blob/main/results/revision-2026-09-04/lifetime-certificate-v2.csv`; the checker is `scripts/check_lifetime_certificate.py`.

Table 5: Main validation items for the proposed v2 implementation

Validation	Acceptance condition	Result
Sketch collision	2^{100} and $2^{100}+1$ take full-precision comparison and return correct order	PASS
Fixed inputs	Byte-identical PK/SK/signed messages on all 12 inputs	12/12
Official-input differential conformance	Ordinary/low-memory APIs agree bytewise; two separate-process repeats	100/100 $\times 2$
Memory safety	No warnings in targeted ASan and UBSan tests	PASS
Radix/level	Targeted RADIX64/RADIX32 and Level-I/III/V tests	PASS
Arm ABI	Candidate, phase, and operation regions match specified byte sizes	PASS
Stack policy	Zero VLAs/`alloca` and dynamic-frame records in target code	PASS
All RP2350 operations	All operations succeed on two boots of the same UF2/input; guards and erasure pass	2/2
No Verify randomness	Random-generator state unchanged across Verify	PASS

The three responses from two low-memory API runs and one ordinary API run agree byte for byte, with SHA-256 `1d86d15c5d2bfbef99f17634496086a3ab80f0e848d34d3e9409d75f3019dd68`. Across the 100 observed inputs, we detect no output difference from changing workspace ownership. However, Compact fixed-precision KeyGen consumes randomness differently from the historical GMP implementation, so archived historical NIST-v2 responses cannot serve as expected outputs. Our claim is differential conformance between ordinary and caller-owned paths within the same proposed v2 algorithm on 100 official inputs, not known-answer agreement with historical GMP responses.

For v3, we use official `p324_3` known answers and apply identical checks to unmodified and adapted implementations. The target KAT in Table 6 is not merely verification of self-generated signatures by the same implementation. It requires byte agreement with public keys, secret keys, signed messages, and recovered messages in official vectors.

Table 6: Functional validation of SQIsign v3.0 `p324_3`

Validation	Official v3	Adapted v3
Host NIST API test	PASS	PASS
Host self-test	PASS	PASS
Host official KAT	PASS	PASS
RP2350 official KAT	PASS	PASS
Modified-signature rejection	PASS	PASS

We separately record the operation arena body, guarded shared operation region, link-time main-SRAM-bank reservation, PSP/MSP reservations, and measured stack watermarks. We measure stack usage by prefilling the stack with a known pattern and measuring the extent overwritten during execution (the stack watermark). This measured extent is not a worst-case stack bound. Unreserved memory is computed by subtracting full PSP and MSP reservations from total SRAM, not observed watermarks.

6 Results

For v2, we evaluate how arena reduction translates into whole-firmware SRAM reservation, alongside stack, flash, and execution time. For v3, we report local storage-sharing effects and analyze stack bounds in a separately built fixed-array variant. Finally, we evaluate residual input dependencies in both versions.

6.1 RAM, stack, flash, and execution-time trade-offs

Table 7 shows changes from the full-norm-table implementation to proposed v2. Removing two full-precision norm rows shrinks candidate-search storage to 14,024 bytes. Phase-shared storage nevertheless remains 94,912 bytes because the existing ML2 retry region becomes the largest member. The 77,168-byte lattice state remains live during candidate search and does not share its storage. Explaining this change in the largest member requires examining simultaneous lifetimes, not only local array sizes.

Table 7: Cortex-M33/RADIX32 arena changes from the bit-length and leading-bit pair

Item	Full table [B]	Proposed v2 [B]	Change [B]
Two resident full-norm rows	269,568	0	−269,568
Candidate-search region	275,840	14,024	−261,816
Phase-shared region	275,840	94,912	−180,928
Whole operation arena	353,008	172,080	−180,928

The whole-operation-arena reduction is

$$\frac{353{,}008 - 172{,}080}{353{,}008} = 0.51253229 \tag{10}$$

or 51.2532%.

Table 8 reports the linked proposed v2 implementation. The 180,928-byte reduction from the full-table implementation translates directly into reduced total SRAM reservation and increased unreserved SRAM. It is therefore not merely a transfer of heap data to stack. The proposed v2 binary image occupies 288,384 bytes, 880 bytes more than the full-table implementation, due to the code for full-precision regeneration and comparison.

Table 8: Memory reserved to execute proposed v2 on RP2350

Item	Proposed v2 [B]	Notes
On-chip SRAM total	532,480	Main and auxiliary banks
Operation arena body	172,080	KeyGen/Sign/Verify shared region
Guarded operation region	172,208	Main bank
PSP reservation	131,072	Main bank
Other main-bank regions	9,936	Data, runtime state, etc.
Main-bank linked use	313,216	Total of the above
MSP reservation	8,192	Auxiliary bank
Total SRAM reservation	321,408	Relative to full table: −180,928
Unreserved SRAM	211,072	Relative to full table: +180,928
Heap	0	No dynamic-allocation symbols
`.text`	166,092	XIP flash
`.rodata`	116,232	XIP flash
`.data`	5,984	Loaded from flash into RAM
`.bss`	306,960	RAM, including reservations
Binary image	288,384	Relative to full table: +880

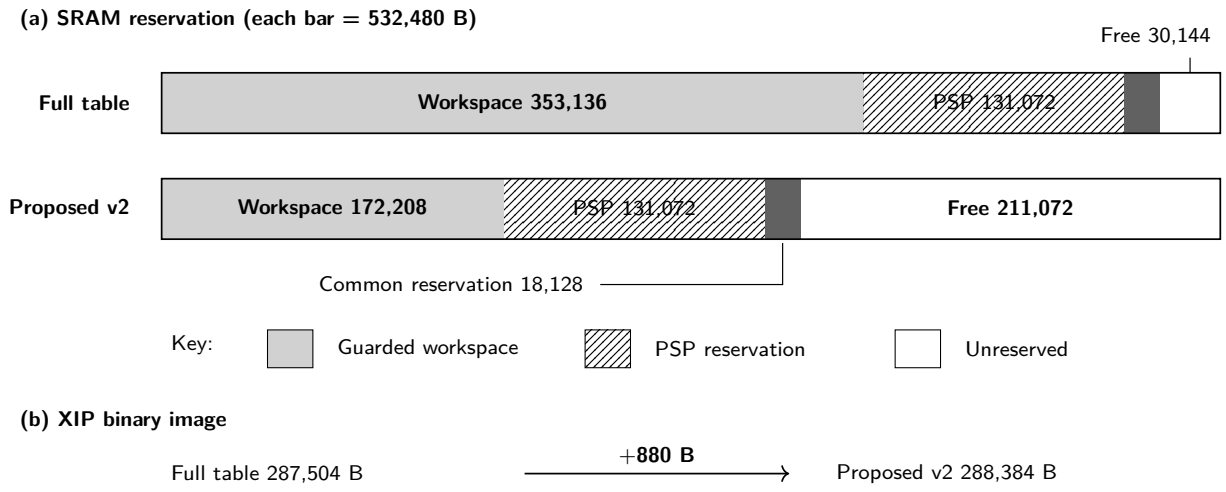

Figure 4: RP2350 SRAM reservations of the full-table and proposed v2 implementations. Both bars represent the total 532,480 bytes of on-chip SRAM on the same scale. The shared operation region includes 128 guard bytes. The 18,128-byte common reservation comprises 8,192 MSP bytes and 9,936 other main-bank bytes. PSP and MSP denote full reservations, not measured watermarks; only white regions are unreserved. The binary-image difference at lower right is not included in the SRAM bars.

We reserve 131,072 bytes for PSP. Two boots using the same UF2 and deterministic input observe watermarks of 91,980 bytes for KeyGen, 120,452 for Sign, and 20,768 for Verify. Both boots give identical values. Even for Sign, the deepest observation leaves 10,620 bytes between watermark and reservation. MSP observations also agree across both boots, with a conservatively estimated watermark of 2,396 bytes against an 8,192-byte reservation.

Two observations alone do not bound input-dependent execution paths. We therefore separately analyze a rebuilt image whose `.text` section is byte-identical to the measured ELF. Its 1154 records in 126 stack-usage files contain no dynamic frames. For functions reachable from linked KeyGen, Sign, and Verify, we supply all missing stack information and resolve all indirect-call targets, leaving no unresolved records. For two recursive routines, the source condition that each child call halves the input length, together with maximum length 248, bounds simultaneously live frames by 9. The check stops if source digests, generated constants, indirect-call resolution, branch veneers, or handwritten-assembly/newlib frame information change.

Table 9 adds the Armv8-M Secure-state maximum exception-entry allowance of 212 bytes to the longest paths of the recursion-collapsed call graph. All three operations fit within the PSP reservation, with the smallest margin, for Sign, being 10,408 bytes. Agreement between measurements and software-call bounds confirms correspondence between the analyzed ELF and measured paths.

Table 9: PSP analysis of the fixed v2 firmware image. PSP bounds add the maximum 212-byte exception-entry allowance to software-call bounds. Reservation margins are differences from the 131,072-byte PSP reservation.

Operation	Observed [B]	Software calls [B]	PSP bound [B]	Margin [B]
KeyGen	91,980	91,980	92,192	38,880
Sign	120,452	120,452	120,664	10,408
Verify	20,768	20,768	20,980	110,092

The analysis covers KeyGen, Sign, and Verify Thread-mode paths linked into the fixed firmware image and one maximum exception entry stacked on PSP. It excludes interrupt-handler software frames, the enabled interrupt-request set, interrupt nesting, and MSP usage at exception entry. We can therefore state that the linked reachable operation-PSP paths fit their reservation, but not a whole-program worst-case stack bound including interrupts. The 211,072-byte unreserved value likewise deducts full PSP/MSP reservations rather than measurements.

Table 10 reports elapsed times from the first boot. Cycle counts are converted from elapsed time assuming a constant 150 MHz clock during the operation, not read directly from a performance counter. On the second boot, KeyGen, Sign, and Verify take 2,698.150324, 7,611.257377, and 0.814356 s, respectively. Differences from the first boot are 295, −1150, and 15 μs. This demonstrates only reproducibility of the same deterministic path, not performance over an input distribution. KeyGen takes approximately 45 minutes and Sign

approximately 127 minutes. The implementation demonstrates memory feasibility, not real-time signing.

Table 10: Deterministic KeyGen, Sign, and Verify on RP2350 (first boot; the second boot has identical PSP watermarks)

Operation	Time [s]	Cycles at 150 MHz	PSP watermark [B]
KeyGen	2,698.150029	404,722,504,350	91,980
Sign	7,611.258527	1,141,688,779,050	120,452
Verify	0.814341	122,151,150	20,768

We conduct two host tests reversing the execution order of full-table and proposed v2 implementations. Each test comprises 30 paired measurements: both implementations run twice on each of 15 seeds. The proposed/full-table median-time ratios are 1.003695 and 1.006464 for KeyGen, and 0.999481 and 0.998980 for Sign. Host KeyGen increases by less than 1%, while both Sign ratios are below 1. These are descriptive statistics for finite measurement sets, not tests establishing general speedup or equivalence. Ratios of single target observations are 1.000704 for KeyGen, 1.037312 for Sign, and 1.000593 for Verify. Without a repeated target distribution, we do not conclude that Sign is generally 3.7% slower.

Relative to the full-table implementation, proposed v2 reduces operation RAM by 180,928 bytes, while increasing local stack frames by 48–64 bytes and flash usage by 880 bytes. Host KeyGen shows a small increase in execution time, whereas no Sign time increase is confirmed. Absolute target runtimes are long, and repeated measurements needed to estimate a slowdown rate are absent. We therefore do not position proposed v2 as superior on every metric. It trades small stack and flash increases for a substantial reduction in operation RAM.

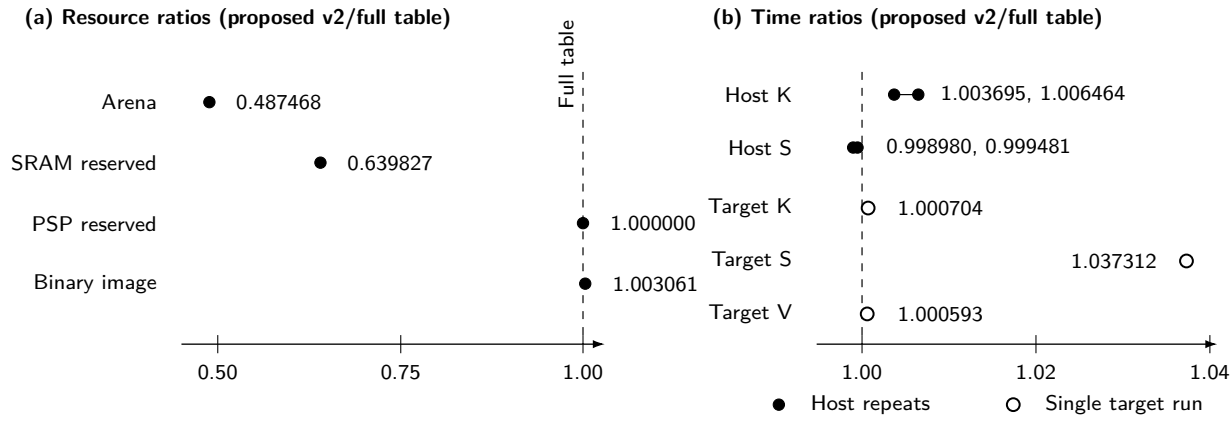

Figure 5: Proposed v2 RAM, stack-reservation, flash, and execution-time ratios normalized to the full-table implementation. Memory figures come from link results and type sizes. Host times show the two 30-pair tests with reversed execution order as separate points. Target times are open markers because each is a single observation. Points from different conditions are not connected. PSP reservation is unchanged, although local stack frames increase by 48–64 bytes.

6.2 Results of transfer to SQIsign v3

The implementation modifying one function applies two overlays only to `quat_lll_dual_reduce_ideal`. Table 11 compares it with unmodified v3 built from the same official baseline commit. Both implementations process 100 official responses each on the host. The adaptation reduces this function's stack frame from 34,816 to 30,888 bytes. The 3,928-byte difference agrees with the reduction in Sign PSP watermark in all five repetitions. This repeated test examines whether local function-frame reduction translates into target stack usage. Transfer to two functions is evaluated using the implementation modifying two functions below.

Table 11: Official SQIsign v3.0 and the implementation modifying one function on RP2350. Time columns are medians of five runs each; changes are medians of the five individual paired ratios.

Item	Official v3	1 function	Change
Largest fixed stack frame [B]	34,816	30,888	−3,928 (−11.282%)
KeyGen PSP watermark [B]	56,972	56,972	0
Sign PSP watermark [B]	101,060	97,132	−3,928 (−3.8868%)
Verify PSP watermark [B]	37,444	37,444	0
Cryptographic-library instructions [B]	204,157	203,945	−212 (−0.1038%)
KeyGen time [s]	2.263654	2.269166	+0.2416%
Sign time [s]	7.152690	7.182418	+0.4190%
Verify time [s]	0.822142	0.820780	−0.1713%

Unchanged KeyGen and Verify PSP watermarks are consistent with the implementation modifying one function modifying only Sign. All three operation watermarks remain constant across five repetitions in both implementations. Paired Sign-time increases range from 0.4112–0.4204%, with median 0.4190%. This observation applies to the specific binary, board, clock, and deterministic known-answer vector. It is neither an input-distribution estimate nor a slowdown rate for the implementation modifying two functions.

The implementation modifying two functions adds the third overlay in `protocols_sign`. RP2350 measurements in Table 12 are bound to source and firmware without uncommitted changes. Vector-0 KeyGen, Sign, and Verify match official responses. Sign PSP watermark decreases from official v3's 101,060 to 96,396 bytes, a total reduction of 4,664 bytes (4.6151%). The additional 736 bytes saved beyond the implementation modifying one function agree with the `protocols_sign` frame reduction. Because this target measurement is performed only once, its elapsed time is not used for performance comparison.

Table 12: Official v3 and the implementation modifying two functions for `p324_3` on RP2350. Function frames are static values from Arm object files. PSP watermarks are medians of five official runs and one observation for the implementation modifying two functions.

Item	Official v3	2 functions	Change
`quat_lll_dual_reduce_ideal` frame [B]	34,816	30,888	−3,928
`protocols_sign` frame [B]	20,008	19,272	−736
KeyGen PSP watermark [B]	56,972	56,972	0
Sign PSP watermark [B]	101,060	96,396	−4,664 (−4.6151%)
Verify PSP watermark [B]	37,444	37,444	0

To test whether the changes depend on one parameter set, we generate official v3 and the implementation modifying two functions for all three official parameter sets. Each configuration passes API tests and scheme self-tests, and its outputs match the official responses for all 100 inputs. This comprises 600 executions checking 300 distinct parameter/input pairs on two implementations. As Table 13 shows, both function frames decrease in all six comparisons. This supports applicability of local storage sharing to the three parameter sets, but does not establish whole-firmware SRAM or target stack watermarks for `p500_27` and `p664_17`.

Table 13: Arm Cortex-M33 stack frames of the implementation modifying two functions across three official parameter sets.

Parameter	Function	Official [B]	Two-function [B]	Reduction
`p324_3`	`quat_lll_dual_reduce_ideal`	34,816	30,888	−3,928 (11.2822%)
`p324_3`	`protocols_sign`	20,008	19,272	−736 (3.6785%)
`p500_27`	`quat_lll_dual_reduce_ideal`	46,088	40,904	−5,184 (11.2480%)
`p500_27`	`protocols_sign`	27,304	26,328	−976 (3.5746%)
`p664_17`	`quat_lll_dual_reduce_ideal`	63,016	55,904	−7,112 (11.2860%)
`p664_17`	`protocols_sign`	36,728	35,392	−1,336 (3.6376%)

For the multi-input test, we generate four firmware images from firmware commit `76f88dcc...`, official source `6d017708...`, and implementation modifying one function `9293313f...`. Every image has a zero-byte heap section, with allocation-function entry points consisting only of three forced-stop handlers. The 19 dynamic-stack records are identical across implementations and layouts. Table 14 reports the results.

Table 14: Local stack-reduction checks using 10 official-response inputs and two code layouts. Each column covers 10 inputs × 2 layouts. Cross-layout PSP comparison covers 10 inputs each for KeyGen, Sign, Verify, and modified-signature rejection, totaling 40 pairs.

Item	Official v3	One-function
Valid KeyGen/Sign/Verify	20/20	20/20
Modified-signature rejection	20/20	20/20
Sign PSP watermark [B]	101,060	97,132
PSP mismatches across layouts A/B	0/40	0/40
Sign median time [s]	7.134887	7.229949
Paired timing-change range [%]	–	1.1327–1.5592

All 40 valid KeyGen/Sign/Verify paths and 40 modified-signature rejections succeed. Operation PSP watermarks agree in all 80 comparisons between layouts A and B for identical implementation/input pairs. The implementation modifying one function saves 3,928 Sign bytes in all 20 observations. Thus, within these 10 inputs and two layouts, the observed reduction does not depend solely on one known-answer input or one link address.

However, Sign-time differences in the multi-vector firmware range from 1.1327–1.5592%, with median 1.3323%, unlike the single-vector firmware's 0.4190%. We therefore do not claim one general slowdown percentage for the implementation modifying one function. Across the two layouts, input-wise Sign times have Pearson correlations of 0.999999867 for official v3 and 0.999999912 for the implementation modifying one function. Maximum/minimum input-time ratios are 1.474635 and 1.468729, respectively. Seeds, keys, and messages change simultaneously across these 10 inputs, so the result cannot be attributed to leakage from a fixed key's secret information.

6.3 Stack-bound analysis using the fixed-array variant

We inspect all 19 dynamic-stack records and implement a fixed-array variant embedding array bounds derived from `p324_3/RADIX32` into generated source. Exceeding a bound invokes a forced stop that remains enabled under `NDEBUG`. Rebuilding with `-Werror=vla -Werror=alloca` converts 18 records into fixed-size frames. The remaining `mp_div_unsigned` is inlined or removed, leaving zero compiler-reported dynamic local frames. We check 100 official responses on the host and vector 0 in a commit-pinned firmware image on RP2350. ELF checks confirm a zero-byte heap section and forced-stop handlers at all three dynamic-allocation entry points.

Table 15: Official v3 and the fixed-array variant. PSP watermarks come from separate measurement series using the same vector 0 and board; they are not used for timing comparison.

Item	Official v3	Fixed-array	Change
Dynamic local-frame records	19	0	−19
Cryptographic-library instructions [B]	204,153	206,297	+2,144
KeyGen PSP watermark [B]	56,972	62,096	+5,124
Sign PSP watermark [B]	101,060	101,060	0
Verify PSP watermark [B]	37,444	40,252	+2,808
Host official responses	–	100/100	–

The fixed-array variant makes frame sizes statically determinable, but does not reduce Sign PSP watermark and increases KeyGen and Verify watermarks. This does not support the expectation that replacing VLAs with fixed arrays alone reduces measured stack usage. Whether the PSP reservation suffices must nevertheless be investigated separately from observed watermarks. We therefore use the fixed-array variant to derive link-specific software-call PSP bounds and add hardware exception-entry costs.

First, we replace `fprintf` and `abort` in newlib diagnostic paths unreachable on successful completion with explicit forced-stop handlers. The cryptographic library is rebuilt with `-fstack-usage -Werror=vla -Werror=alloca`. We then resolve BL/BLX instructions, tail calls to other symbols, and branch veneers in linked disassembly and cross-check all `.su` records. For fixed assembly and newlib helpers without compiler-generated stack information, manual records bind SP-subtraction instructions, maximum saved state, instruction fragments, and function-body SHA-256. Intra-function conditional branches are distinguished from calls. All three conditional branches crossing symbol boundaries are continuations of DCP floating-point wrappers and are included in their manual frames. Missing stack information and unresolved indirect-call targets are thereby reduced to zero for functions reachable from KeyGen, Sign, and Verify.

The call graph contains three recursive strongly connected components for KeyGen and Sign and none for Verify. In discrete-log recursion, each child call halves the input length. With maximum input length 198, we bound simultaneously live frames by 9. Burnikel–Ziegler division halves the multiprecision word count per recursion; for at most 60 RADIX32 words, we use a bound of 11 frames. The analysis stops if source decrease conditions or generated constants change. Weighting recursive components by these depth bounds and computing longest paths in the collapsed call graph gives Table 16.

Table 16: Link-derived PSP bounds for the fixed-array variant. Maximum exception-entry costs are added to software-call bounds; margins are differences from the 131,072-byte PSP reservation.

Operation root	Conservative PSP bound [B]	Margin [B]
KeyGen	108,300	22,772
Sign	127,932	3,140
Verify	40,468	90,604

The RP2350 firmware uses the `rp2350-arm-s` configuration in Armv8-M Secure state. For Secure-state software, entry into a Non-secure exception with active floating-point context can produce a hardware exception frame of up to 52 words. One additional word may be required for 8-byte alignment [27]. We therefore uniformly add 212 bytes to software-call bounds. Across the linked ELF, only the setup and restoration instructions in the operation trampoline execute `MSR PSP`. Handler mode uses MSP. We consequently include the first maximum exception entry stacked on PSP during the operation, while leaving interrupt-handler software frames and nesting as MSP usage requiring separate evaluation.

Executing vector 0 on hardware using fixed-array firmware built from the same commit as the analyzed version gives KeyGen, Sign, and Verify outputs that all match the official responses. Observed PSP watermarks are 62,096, 101,060, and 40,252 bytes, all within the bounds above. Verify in particular observes 40,252 bytes against a 40,256-byte software-call bound. Its distance from the bound including maximum exception entry is 216 bytes. This proximity supports correspondence between analysis and measurement, but does not imply margin for another binary or input.

All three bounds fit their reservations, but Sign's margin is only 3,140 bytes. This is a conservative PSP bound for the fixed commit, `p324_3/RADIX32`, and current compilation/link results, not formal verification of C and assembly semantics. Unresolved indirect callbacks remain on the interrupt side. The enabled interrupt-request set, priorities, nesting, and MSP usage at exception entry are also unevaluated. We therefore claim PSP bounds for linked KeyGen, Sign, and Verify including software calls and maximum exception entry, but not a whole-program worst-case stack bound.

We also record cross-version absolute times for context. On the same RP2350 at 150 MHz, proposed v2's first-boot KeyGen, Sign, and Verify take 2,698.150029, 7,611.258527, and 0.814341 s. The second boot of the same UF2 takes 2,698.150324, 7,611.257377, and 0.814356 s. This shows inter-boot reproducibility for one input, not samples from an input distribution. Official v3's five-run medians are 2.263654, 7.152690, and 0.822142 s. Version 3 KeyGen and Sign are substantially shorter, but parameters, algorithms, integer backends, and implementation maturity differ simultaneously. We do not interpret the difference as the speedup of one memory-reduction transformation. Proposed v2 also has an explicit 172,080-byte arena separate from PSP; PSP watermarks alone do not rank total SRAM usage.

6.4 Side-channel assessment

Detailed attack-surface tracing primarily targets proposed v2. For v3, we perform timing diagnostics over 10 official-response inputs in two code layouts, timing tests comparing 10 keys with fixed message and signing randomness, host control-flow/effective-address diagnostics, and RP2350 timing tests under the same fixed conditions. These tests reproduce key-associated differences. However, secret-key data include the corresponding public key, preventing attribution solely to secret components. The official v3 specification also states that the full submitted implementation is not completely constant-time [5]. We do not evaluate power/EM acquisition or key recovery. Memory reduction and known-answer agreement therefore cannot imply side-channel resistance.

Fixed arena offsets remove address variation due to dynamic allocation, but do not make operation counts or accessed values secret-independent. Proposed v2 retains input-dependent sketch-tie counts, search reexecution counts, invertibility tests, candidate rejection, and termination on first success. The original SQIsign signing path also contains used-word scans, early-exit comparisons, variable-iteration Euclidean algorithms, and modular exponentiation with secret-derived exponents.

Fixed placement alone therefore does not establish side-channel security; the memory-reduced implementation requires independent assessment.

For proposed v2, we measure 64 fixed and 64 random inputs and repeat the test with reversed execution order. Besides the first-order statistic t_1, we compute a second-order statistic t_2 by squaring deviations from each group's center and applying the same Welch test to the resulting values. Table 17 reports the results. This second-order statistic corresponds to centered-moment preprocessing in higher-order nonspecific t-tests [28]. In both runs, the magnitude of t_2 exceeds the commonly used detection threshold $|t| > 4.5$ [28]. Same-input controls are stable, with between-run timing Pearson correlation 0.999929 and Spearman correlation 0.999771. This small preliminary test has 64 samples per group, so even subthreshold results would not establish nonleakage. Both reversed-order runs nevertheless exceed the threshold in the same direction, showing a timing-distribution difference under these conditions.

Table 17: Host fixed-versus-random timing comparisons for proposed v2 Sign

Run	t_1	t_2	Decision
A	3.570	−5.410	Second-order statistic exceeds threshold
B (reversed order)	3.576	−5.374	Threshold exceeded in same direction

Pearson correlations of Sign time with retained pair comparisons, retained pair ties, and search reexecutions have absolute values generally at most 0.217. The measured metrics thus do not identify the use of partial norm storage as the existing dominant leakage source. Weak correlation is not proof of nonleakage, however, and does not justify treating proposed v2 as a side-channel-security improvement.

For v3, we test timing with fixed message, signing randomness, and key-storage address. We select 10 keys from official `p324_3` known answers and copy each to the same buffer. With a fixed 33-byte message and 48-byte signing seed, each key runs 30 times in two different randomized orders. On Darwin/arm64, we measure 1,200 signatures across official builds and builds modifying one function and verify every signature with its corresponding public key. For both implementations, the Spearman correlation coefficient between the two runs' rankings of key-wise median times is at least 0.987879. The fastest-to-slowest key median span is 52.6313–53.5166% of the median across keys. Thus, key-associated timing differences reproduce under these host conditions. This does not demonstrate secret-only attribution, physical leakage, or key recovery.

We test the same hypothesis on RP2350 using a decision rule set before measurement. We copy 10 keys to the same fixed address and invalidate XIP cache immediately before each measurement. With fixed message and signing randomness, we perform two tests with different key orders, repeating each key five times per test. Across official builds and builds modifying one function, we obtain 200 signatures. Besides successful verification of every signature, we check signature digests, canaries, and firmware/source digests. For each implementation, the rule requires between-test Spearman correlation of key-median ranks of at least 0.8. In each test, the fastest-to-slowest median span must be at least 1% of the median across keys and at least ten times the median within-key median absolute deviation (MAD). Both implementations have Spearman correlation at least 1.000000 and key spans of 50.8419–51.3752%, satisfying the rule. Across all 100 samples per implementation, Sign PSP watermarks are 101,060 bytes for official v3 and 97,132 for the implementation modifying one function, reproducing the 3,928-byte difference. Firmware order is fixed as official then modified, so cross-version timing differences are not used as general performance comparisons. This single-board result demonstrates repeatedly observed key-associated timing differences on RP2350. It does not demonstrate separation of public and secret components, RP2350 control-flow/address traces, analog power/EM leakage, key recovery, or resistance.

For the same input design, we use Apple Clang SanitizerCoverage to record control-flow edge identifiers and effective addresses of loads/stores during `crypto_sign` in two independent processes. Each secret key is copied to the same fixed-address buffer before measurement to remove trivial storage-location differences in the known-answer table. Across official builds and builds modifying one function, all 8 same-key control pairs agree in event counts and two sequence digests. Among 36 comparisons of the other 9 keys against key 0, control-flow counts or digests differ in 36/36 pairs, and effective-address counts or digests in 36/36 pairs. Sequences contain up to approximately 70 million control-flow edges and 358 million memory accesses. Only the first 2 million events are retained; complete sequences are compared by counts and two 64-bit digests. Same-key agreement is therefore not formal equivalence. In every different-key pair, event counts themselves differ, so key-associated structural differences are confirmed without relying on digest collision probabilities. The test does not separate public and secret key components and observes host code only. It is not evidence of secret leakage, RP2350 code behavior, physical waveforms, or a successful attack.

In two separate host measurements, we record Sign control-flow edge sequences and effective-address sequences of targeted load/store instructions. Same-seed controls agree in both sequences. Every comparison between the fixed seed and different signing randomness reproducibly differs in both sequences. Operations reached from first differences include multiprecision used-word scans, most-significant-word comparisons, and

random-ideal sampling.

This result shows that control flow in the compiled full signer changes with signing randomness. The directly varied input is signing randomness, however; we do not claim that first differences alone enable recovery of a long-term secret key.

Mukherjee et al. report an SPA attack recovering secret-derived temporary exponents used by modular exponentiation in Cornacchia processing and connecting them to secret-key recovery [13]. Signing with 12 fixed seeds in our implementation observes the `Sign` → `RepresentInteger` → `Cornacchia` → `sqrt_mod_p` path 361 times. Of these, 190 belong to the modulus class directly corresponding to the published exponent relation.

Isolated modular-exponentiation measurements on RP2350 give Pearson correlation 0.999860 between exponent Hamming weight and elapsed time, with regression slope approximately 4.004 ms per additional set bit. This shows strong variable-time behavior on the physical chip. It is timing of a GPIO-delimited local operation, not analog power/EM acquisition or full-signing key recovery.

We measure three local-countermeasure variants: fixed exponent-processing order, fixed multiplication count, and integration of protected square-root processing into Sign. All retain value-dependent differences or variable upper-level iteration counts. Input corpora, sample counts, timers, variances, and implementation time ratios are provided in `experiments/sca/README.md`. These changes therefore do not support full-signing side-channel resistance. Assessing resistance requires full-signing constant-time processing or masking and independently acquired power/EM waveforms. The measurement setup must also be shown capable of detecting the published attack.

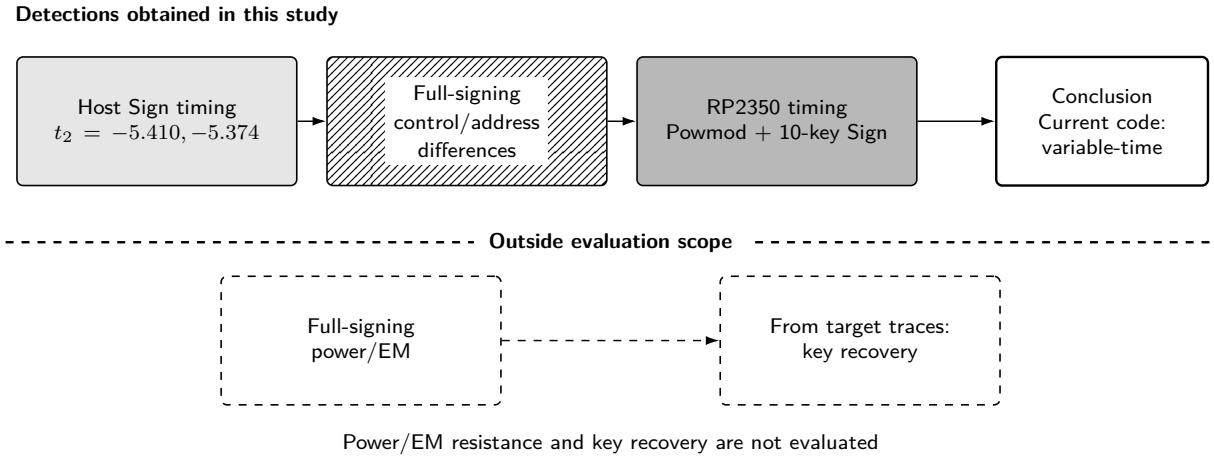

Figure 6: Scope of side-channel assessment. Solid boxes denote detections obtained in this study; dashed boxes denote experiments not performed. We detect variable-time behavior in the current implementation, but do not determine full-signing power/EM resistance or whether keys can be recovered.

7 Discussion and Limitations

7.1 Applicability and implications for embedded integration

The procedure effective in both versions explicitly identifies object lifetimes and interference and reflects them in typed storage placement. Proposed v2 applies it to protocol-wide shared operation storage. The v3 implementation modifying two functions applies the same procedure to two redesigned functions, reducing usage in all six stack-frame comparisons across three parameter sets. In the `p324_3` target test, both function reductions translate into Sign PSP watermark, differing from official v3 by 4,664 bytes. This demonstrates applicability of lifetime-based memory allocation beyond v2 `find_uv`.

The bit-length and leading-bit pair, 1665-bit integer width, 172,080-byte arena, and reachable scope of 52 translation units with allocation functions and VLAs removed are v2-specific. Version 3 lacks the targeted `find_uv` candidate table, and the implementation modifying two functions retains 19 dynamic-stack records. We therefore do not conclude that the concrete v2 memory layout transfers unchanged to v3. What transfers is the formulation and checking procedure for determining shareability from lifetimes. Shared regions and obtained guarantees differ by version.

The concrete memory quantities depend on each version's Level-I-equivalent parameters and RP2350. The same ideas may nevertheless apply to implementations satisfying the following conditions.

1. Large intermediates can be reconstructed from inputs or small regeneration metadata.
2. Computation can be divided into phases with an API that does not retain workspace pointers across phases.
3. Array bounds and alignment can be fixed at compile time for each parameter set.
4. Comparisons can return to full precision when representative values do not determine the order.

The comparison method determines the order when bit lengths or leading bits differ and returns to original full precision when both agree. This may apply beyond norms to search or pruning procedures whose compared values can be reconstructed at full precision from small inputs. Data reduction is demonstrated only for v2 norms, however. Lifetime-based memory allocation yields results for v2 shared operation storage and two v3 functions across three parameter sets, but not for other post-quantum schemes or microcontrollers. Applicability beyond SQIsign is therefore a hypothesis requiring future validation.

Proposed v2 leaves 211,072 bytes unreserved, increasing integration headroom relative to the full-table implementation's 30,144 bytes. This may support production randomness, communication buffers, interrupt stacks, additional countermeasure state, or other parameter sets. Unreserved capacity does not show that all these fit simultaneously. After constant-time processing or masking is introduced, RAM, stack, flash, and execution time must be remeasured to assess trade-offs against this implementation. For adapted v3, only local stack reductions are measured; total SRAM reservation and unreserved SRAM under the same definition have not been computed.

7.2 Validity and remaining constraints

The results have the following limitations.

1. **Target scope**: hardware measurements are limited to RP2350, one Arm Cortex-M33 core, and Level I/RADIX32 for v2. Host functional tests and Arm frame audits of the v3 implementation modifying two functions cover three parameter sets, but full firmware builds and hardware measurements cover only `p324_3/m4f`. Results correspond to the commits stated here and do not establish transfer to other microcontrollers or implementation versions.
2. **Input scope**: v2 target measurements repeat identical deterministic KeyGen, Sign, and Verify over two boots using reproducibility CTR-DRBG. The v3 implementation modifying one function has five reversed-order single-vector pairs and ten vectors in two layouts. The two-function target measurement is one vector-0 run. These tests do not characterize input distributions, board variation, high-retry inputs, or two-function timing distributions. No production random generator incorporating entropy sources, reseeding, and fault handling is integrated.
3. **Functional equivalence**: v2 checks 12 fixed inputs and ordinary/low-memory API differences on 100 official inputs. Version 3 processes 300 distinct parameter/input pairs on two implementations, totaling 600 official-response checks. These finite-corpus regression tests do not prove all-input C-program equivalence. The v2 differential oracle is the same commit's ordinary API, not historical GMP responses.
4. **Manual conditions**: the annotation-driven placement generator and coverage checks detect missing annotations for declared members and direct accesses. Phases, lifetimes, indirect aliases, pointer retention, and failure-path erasure remain manually specified. ASan/UBSan, canaries, ELF audits, and official responses provide separate error-detection routes, but do not prove every C behavior including aliases.
5. **Stack bounds**: v2 bounds cover input-dependent Thread-mode paths of a fixed firmware image and one maximum exception entry. The v3 fixed-array variant is a separate binary with zero dynamic-frame records; its bounds do not apply to the implementation modifying two functions retaining 19 such records. No missing stack information is found among direct callees of four candidate interrupt entry functions. However, 18 indirect-callback sites, enabled interrupt requests, priorities, nesting, handler frames, and MSP use at exception entry are unevaluated. Neither result is a whole-program bound including interrupts.
6. **Security scope**: v2 signing retains input-dependent branches, memory addresses, retries, and recomputation. We also observe v3 key-associated timing and control-flow/address differences, but public and secret components change simultaneously, precluding secret-only attribution. Full-signing power/EM waveforms, independent acquisition, attack reproduction, masking, and RP2350 structural traces are unevaluated. Side-channel resistance is therefore not demonstrated.

7. **Practicality**: v2 KeyGen and Sign are too slow for many product applications. The 211,072 unreserved bytes are integration headroom, not demonstrated simultaneous capacity for production randomness, communication, interrupts, and countermeasure state. A v3 total SRAM reservation under the same definition is also unavailable.

Table 18 distinguishes obtained stack evidence from excluded elements for each implementation.

Table 18: Scope of stack evaluation. The fixed-array and implementation modifying two functionss are different binaries.

Target	Obtained evidence	Excluded elements
v2 KeyGen/Sign/Verify PSP	Bound on linked input-dependent Thread-mode paths plus one maximum exception entry	Interrupt handlers, nesting, MSP use at exception entry
v3 fixed-array operation PSP	Bound on linked software calls plus one maximum exception entry	Application to implementation modifying two functions; handlers and MSP
v3 operation PSP (two functions modified)	Local frame differences in two functions and one `p324_3` watermark	Operation-path bound including 19 dynamic frames
Interrupt-side stack	MSP reservation/watermark; direct-callee stack-information audit	Enabled interrupts, indirect callbacks, nesting, MSP use at exception entry

8 Conclusion

For SQIsign v2.0.1 Level I, we combine typed lifetime-based memory allocation with partial norm storage and recomputation to reduce the operation arena from 353,008 to 172,080 bytes while preserving the original full-precision comparison order. The RP2350 firmware uses no GMP, dynamic allocation, or external PSRAM. It reserves a total of 321,408 SRAM bytes and leaves 211,072 bytes unreserved. Differential conformance on 100 official inputs, lifetime audit tables, ELF audits, and fixed-image PSP analysis support the functional and memory results within this scope.

For SQIsign v3.0, we apply the same placement principle to `quat_lll_dual_reduce_ideal` and `protocols_sign`, reducing six Arm frames across three parameter sets. On `p324_3` hardware, Sign PSP watermark decreases from 101,060 to 96,396 bytes. The bit-length and leading-bit pair remains v2-specific. Fixed-array PSP bounds also do not apply to the implementation modifying two functions or interrupt-handler MSP.

Both v2 and v3 retain input-dependent execution time, control flow, and memory addresses. Version 2 KeyGen and Sign also remain impractically slow. Our results therefore concern low-memory placement and its checking methodology. They do not establish constant-time behavior, physical-leakage resistance, whole-program worst-case stack bounds, or production readiness.

Artifacts and Reproducibility

Supplementary artifacts, including source code, reconstruction patches, experiment scripts, binary digests, and target captures, are public at `https://github.com/quantumshiro/tinysqisign`.

References

[1] G. Alagic, M. Bros, P. Ciadoux, Q. Dang, T. H. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, H. Silberg, D. Smith-Tone, and N. Waller. *Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process*. Tech. rep. NIST IR 8610. National Institute of Standards and Technology, May 2026. DOI: 10.6028/NIST.IR.8610. URL: https://doi.org/10.6028/NIST.IR.8610.

[2] National Institute of Standards and Technology. *Round 3 Additional Digital Signature Schemes*. 2026. URL: https://csrc.nist.gov/projects/pqc-dig-sig/round-3-additional-signatures (visited on 09/04/2026).

[3] L. De Feo, D. Kohel, A. Leroux, C. Petit, and B. Wesolowski. "SQISign: Compact Post-Quantum Signatures from Quaternions and Isogenies". In: *Advances in Cryptology – ASIACRYPT 2020*. Vol. 12491. Lecture Notes in Computer Science. Springer, 2020, pp. 64–93. DOI: 10.1007/978-3-030-64837-4_3. URL: https://doi.org/10.1007/978-3-030-64837-4_3.

[4] SQIsign submission team. *SQIsign: Algorithm Specifications and Supporting Documentation, Version 2.0.1*. July 2025. URL: https://sqisign.org/spec/sqisign-20250707.pdf (visited on 09/04/2026).

[5] SQIsign submission team. *SQIsign: Algorithm Specifications and Supporting Documentation, Version 3.0*. Tech. rep. National Institute of Standards and Technology, Sept. 2026. URL: https://sqisign.org/spec/sqisign-20260901.pdf (visited on 09/04/2026).

[6] W. Kim, J. Lee, H. Kim, and C. Lee. "SQIsign with Fixed-Precision Integer Arithmetic". In: *Public-Key Cryptography – PKC 2026*. Vol. 16553. Lecture Notes in Computer Science. Earlier version: Cryptology ePrint Archive, Paper 2025/1649. Springer, 2026, pp. 3–30. DOI: 10.1007/978-3-032-26737-5_1. URL: https://doi.org/10.1007/978-3-032-26737-5_1.

[7] W. Kim, C. Lee, and H. Yoo. "Compact Quaternion Algorithms for SQIsign". In: *Cryptology ePrint Archive, Paper 2026/1031* (2026). URL: https://eprint.iacr.org/2026/1031.

[8] M. J. Kannwischer, M. Krausz, R. Petri, and S.-Y. Yang. "pqm4: Benchmarking NIST Additional Post-Quantum Signature Schemes on Microcontrollers". In: *Cryptology ePrint Archive, Paper 2024/112* (2024). URL: https://eprint.iacr.org/2024/112.

[9] F. Carvalho Rodrigues, D. Gazzoni Filho, G. Adj, I. A. Canales-Martínez, J. Chávez-Saab, J. López, M. Scott, and F. Rodríguez-Henríquez. "Generation of Fast Finite Field Arithmetic for Cortex-M4 with ECDH and SQIsign Applications". In: *Cryptology ePrint Archive, Paper 2025/1322* (2025). URL: https://eprint.iacr.org/2025/1322.

[10] M. A. Aardal, G. Adj, A. Alblooshi, D. F. Aranha, I. A. Canales-Martínez, J. Chávez-Saab, D. L. Gazzoni Filho, K. Reijnders, and F. Rodríguez-Henríquez. "Optimized One-Dimensional SQIsign Verification on Intel and Cortex-M4". In: *Cryptology ePrint Archive, Paper 2024/1563* (2024). URL: https://eprint.iacr.org/2024/1563.

[11] L. De Feo, L.-J. Jian, T.-Y. Wang, and B.-Y. Yang. "SQISign on ARM". In: *Cryptology ePrint Archive, Paper 2026/394* (2026). URL: https://eprint.iacr.org/2026/394.

[12] SQIsign submission team. *The SQIsign Implementation, Version 3.0*. Git tag nist-v3; frozen revision 6d017708... Sept. 2026. URL: https://github.com/SQISign/the-sqisign/tree/nist-v3 (visited on 09/04/2026).

[13] A. Mukherjee, M. Czuprynko, D. Jacquemin, P. Kutas, and S. Sinha Roy. "Simple Power Analysis Attack on SQIsign". In: *Progress in Cryptology – AFRICACRYPT 2025*. Vol. 15651. Lecture Notes in Computer Science. Earlier version: Cryptology ePrint Archive, Paper 2025/830. Springer, 2025, pp. 245–269. DOI: 10.1007/978-3-031-97260-7_12. URL: https://doi.org/10.1007/978-3-031-97260-7_12.

[14] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein. "Register Allocation via Coloring". In: *Computer Languages* 6.1 (1981), pp. 47–57. DOI: 10.1016/0096-0551(81)90048-5.

[15] J. Gergov. "Algorithms for Compile-Time Memory Optimization". In: *Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms*. ACM/SIAM, 1999, pp. 907–908. DOI: 10.5555/314500.315082.

[16] G. Borin, M. Corte-Real Santos, J. Komada Eriksen, R. Invernizzi, M. Mula, S. Schaeffler, and F. Vercauteren. "Qlapoti: Simple and Efficient Translation of Quaternion Ideals to Isogenies". In: *Cryptology ePrint Archive, Paper 2025/1604* (2025). URL: https://eprint.iacr.org/2025/1604.

[17] Y.-F. Lai. "Qlapoti+ and More: Optimizing Isogeny-based Signatures". In: *Cryptology ePrint Archive, Paper 2026/1640* (2026). URL: https://eprint.iacr.org/2026/1640.

[18] M. Duparc, A. Leroux, and S. Schaeffler. "Qlapoty: Improved Analysis and Efficiency for Quaternionic Ideal to Isogeny Transformation". In: *Cryptology ePrint Archive, Paper 2026/1700* (2026). URL: https://eprint.iacr.org/2026/1700.

[19] Y. Hu, S. Shen, H. Yang, and W. Wang. "Paving the Way for SQIsign: Toward Efficient Deployment on 32-bit Embedded Devices". In: *Mathematics* 12.19 (2024), p. 3147. DOI: 10.3390/math12193147. URL: https://doi.org/10.3390/math12193147.

[20] W. Wang, C. Wang, Y. Wu, Q. Xue, J. Zheng, and Y. Zhao. "Exposing SIMD Parallelism in SQIsign: An AVX-512 Implementation". In: *arXiv preprint arXiv:2608.13948* (2026). DOI: 10.48550/arXiv.2608.13948. URL: https://arxiv.org/abs/2608.13948.

[21] Anchorage Digital. *sqisign-rs: A Pure Rust Implementation of SQIsign v2.0*. Repository release 0.5.0. 2026. URL: https://github.com/anchorageoss/sqisign-rs (visited on 09/04/2026).

[22] F. Kouider, A. Mukherjee, D. Jacquemin, and P. Kutas. "Constant-time Integer Arithmetic for SQIsign". In: *Cryptology ePrint Archive, Paper 2025/832* (2025). URL: https://eprint.iacr.org/2025/832.

[23] O. Hanyecz, A. Karenin, E. Kirshanova, P. Kutas, and S. Schaeffler. "Constant Time Lattice Reduction in Dimension 4 with Application to SQIsign". In: *Cryptology ePrint Archive, Paper 2025/027* (2025). URL: https://eprint.iacr.org/2025/027.

[24] A. Basso, C. He, D. Jacquemin, F. Kouider, P. Kutas, A. Mukherjee, S. Schaeffler, and S. Sinha Roy. "Constant-time Quaternion Algorithms for SQIsign". In: *Cryptology ePrint Archive, Paper 2025/2192* (2025). URL: https://eprint.iacr.org/2025/2192.

[25] Raspberry Pi Ltd. *RP2350 Datasheet: A Microcontroller by Raspberry Pi*. RP-008373-DS-2. 2025. URL: https://pip-assets.raspberrypi.com/categories/1214-rp2350/documents/RP-008373-DS-2-rp2350-datasheet.pdf?disposition=inline (visited on 09/04/2026).

[26] T. Granlund and the GMP development team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. 6.3.0. 2023. URL: https://gmplib.org/gmp-man-6.3.0.pdf (visited on 09/04/2026).

[27] J. Yiu. *How Much Stack Memory Do I Need for My Arm Cortex-M Applications?* Arm. Mar. 2016. URL: https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/how-much-stack-memory-do-i-need-for-my-arm-cortex--m-applications (visited on 09/04/2026).

[28] T. Schneider and A. Moradi. "Leakage Assessment Methodology: A Clear Roadmap for Side-Channel Evaluations". In: *Journal of Cryptographic Engineering* 6.2 (2016), pp. 85–99. DOI: 10.1007/s13389-016-0120-y. URL: https://eprint.iacr.org/2015/207.